UQA Reference Guide

UQA Reference Guide

1. Introduction
    - 1.1 What is UQA?
    - 1.2 Design Principles
    - 1.3 Project Status
2. Theoretical Foundations
    - 2.1 Paper 1: A Unified Mathematical Framework for Query Algebras Across Heterogeneous Data Paradigms (December 2023)
    - 2.2 Paper 2: Extending the Unified Mathematical Framework to Support Graph Data Structures (December 2024)
    - 2.3 Paper 3: Bayesian BM25 — A Probabilistic Framework for Hybrid Text and Vector Search (January 2026)
    - 2.4 Paper 4: From Bayesian Inference to Neural Computation (February 2026)
    - 2.5 Paper 5: Vector Scores as Likelihood Ratios — Index-Derived Bayesian Calibration for Hybrid Search (March 2026)
3. Core Concepts
    - 3.1 The Posting List
    - 3.2 Operators
    - 3.3 Scoring
    - 3.4 Multi-Signal Fusion
4. Function Reference
    - 4.1 WHERE-Clause Functions
        - `text_match(field, query)`
        - `bayesian_match(field, query)`
        - `knn_match(field, vector, k)`
        - `traverse_match(start, label, max_hops)`
        - `path_filter(path, value)` / `path_filter(path, op, value)`
        - `spatial_within(field, POINT(x, y), distance_meters)`
        - `vector_exclude(field, positive_vector, negative_vector, k, threshold)`
        - `sparse_threshold(signal, threshold)`
        - `multi_field_match(field1, field2, ..., query[, w1, w2, ...])`
        - `bayesian_match_with_prior(field, query, prior_field, prior_mode)`
        - `temporal_traverse(start, label, hops, timestamp[, source])` / `temporal_traverse(start, label, hops, from_ts, to_ts[, source])`
        - `message_passing(k_layers, aggregation, property_name)`
        - `graph_embedding(dimensions, k_layers)`
    - 4.2 Fusion Meta-Functions
        - `fuse_log_odds(signal1, signal2, ...[, alpha])`
        - `fuse_mean(signal1, signal2, ...)`
        - `fuse_prob_and(signal1, signal2, ...)`
        - `fuse_prob_or(signal1, signal2, ...)`
        - `fuse_prob_not(signal)`
        - `fuse_attention(signal1, signal2, ...)`
        - `fuse_learned(signal1, signal2, ...)`
        - `staged_retrieval(signal1, k1, signal2, k2, ...)`
    - 4.3 SELECT Scalar Functions
        - `path_agg(path, func)`
        - `path_value(path)`
        - `uqa_highlight(field, query [, start_tag, end_tag [, max_fragments, fragment_size]])`

- `uqa_facets(field1 [, field2, ...])`
- `POINT(x, y)`
- `ST_Distance(point1, point2)`
- `ST_Within(point1, point2, distance_meters)` / `ST_DWithin(...)`

4.4 FROM-Clause Table Functions

- `traverse(start, label, max_hops[, source])`
- `rpq(path_expr, start[, source])`
- `temporal_traverse(start, label, hops, timestamp[, source])` / `temporal_traverse(start, label, hops, from_ts, to_ts[, source])`
- `text_search(query, field, table)`
- `generate_series(start, stop[, step])`
- `unnest(array)`
- `regexp_split_to_table(string, pattern [, flags])`
- `json_each(json)` / `json_each_text(json)`
- `json_array_elements(json)` / `json_array_elements_text(json)`
- `create_graph(name)` / `drop_graph(name)`
- `graph_create_node(graph, label[, properties_json])` / `graph_create_edge(graph, type, src, tgt[, properties_json])`
- `graph_nodes(graph[, label][, filter_json])`
- `graph_neighbors(graph, id[, type][, direction][, depth])`
- `graph_traverse(graph, id[, types][, direction][, depth[, strategy]])`
- `graph_edges(graph[, type][, filter_json])` (graph-wide mode)
- `graph_edges(graph, vertex_id[, type | NULL][, direction])` (per-vertex mode)
- `graph_delete_node(graph, id)` / `graph_delete_edge(graph, id)`
- `cypher(graph_name, query) AS (columns)`
- `create_analyzer(name, config)` / `drop_analyzer(name)` / `list_analyzers()`
  - Tokenizers
  - Token Filters
  - Character Filters
  - Built-in Analyzer Presets
  - JSON Configuration Format
- `set_table_analyzer(table, field, analyzer[, phase])`

5. Architecture
   - 5.1 High-Level Architecture
   - 5.2 Storage Layer
   - 5.3 SQL Compiler
   - 5.4 Query Optimizer
   - 5.5 Execution Engine
6. Usage
   - 6.1 Installation
   - 6.2 Creating an Engine
   - 6.3 SQL API
     - Schema Definition
     - Data Manipulation
     - Relational Queries
     - Full-Text Search
     - Vector Search
     - Graph Queries
     - Graph Query Language (openCypher)
     - Multi-Signal Fusion
     - Hierarchical Data
     - Query Introspection
     - Foreign Data Wrappers
   - 6.4 QueryBuilder Fluent API

6.5 Arrow / Parquet Export
SQL API
QueryBuilder API
6.6 Programmatic Document API
6.7 Deep Learning Methods
Layer Spec Types
`Engine.deep_learn()`
`Engine.deep_predict()`
Model Catalog Methods
6.8 Interactive SQL Shell
6.9 Transactions
7. Significance
7.1 Unification Through Algebra
7.2 Closing the Probabilistic Relevance Gap
7.3 Neural Structure from Probability
8. Examples
Fluent API (`examples/fluent/`)
SQL (`examples/sql/`)
9. Testing
10. References

1. Introduction

1.1 What is UQA?

UQA (Unified Query Algebra) is a database prototype that unifies five traditionally isolated data paradigms — relational SQL, full-text search, vector search, graph queries, and geospatial — under a single algebraic structure, using **posting lists** as the universal abstraction.

Where conventional systems force users to stitch together a relational database, a search engine, a vector store, and a graph database — each with its own query language, data model, and operational overhead — UQA provides a single engine that speaks SQL enriched with cross-paradigm functions.

```sql
SELECT title, _score FROM papers
WHERE fuse_log_odds(
    text_match(title, 'attention'),
    knn_match(embedding, ARRAY[0.1, 0.2, ...], 5),
    traverse_match(1, 'cited_by', 2)
) AND year >= 2020
ORDER BY _score DESC;
```

This single query combines BM25 text scoring, KNN vector similarity, and graph traversal reachability into a fused relevance score, then applies a relational filter on publication year — an operation that would require coordinating three separate systems in a conventional stack.

1.2 Design Principles

UQA is built on three principles:

1. **Algebraic unification.** Every paradigm — relational filtering, term-based retrieval, vector similarity, graph traversal — produces the same data structure: an ordered posting list.

Boolean algebra (union, intersection, complement) operates uniformly across all of them.

2. **Probabilistic scoring.** Relevance signals are not raw scores but calibrated probabilities in $[0, 1]$. This enables principled multi-signal fusion through Bayesian inference rather than ad-hoc weighting.
3. **SQL as the interface.** Standard SQL is extended with domain-specific functions (`text_match`, `knn_match`, `traverse`, `fuse_log_odds`) rather than replaced by a novel query language. Users retain the full power of relational SQL — joins, subqueries, CTEs, window functions, transactions — while gaining access to cross-paradigm operations.

1.3 Project Status

UQA is a research prototype implementing the theoretical framework described in a series of four papers (see Section 2). It includes a complete SQL compiler, SQLite-backed persistence, a cost-based query optimizer, parallel execution, and disk spilling. It is designed as a proof-of-concept for the unified query algebra rather than a production database.

2. Theoretical Foundations

UQA is grounded in four papers that progressively build the mathematical framework.

2.1 Paper 1: A Unified Mathematical Framework for Query Algebras Across Heterogeneous Data Paradigms (December 2023)

This foundational paper establishes **posting lists** as a universal abstraction for result sets across relational, text retrieval, and vector search paradigms. A posting list is an ordered sequence of (document_id, payload) pairs, where payloads carry paradigm-specific metadata — field values for relational queries, term positions and scores for text retrieval, similarity scores for vector search.

Key results:

- **Boolean algebra.** The set of posting lists under union, intersection, and complement forms a complete Boolean algebra isomorphic to the power set of documents (Theorem 2.1.2). This means any combination of cross-paradigm queries can be composed using the same three operations.
- **Operator calculus.** Primitive operators — T (TermRetrieval), V_θ (VectorSimilarity), KNN_k, Filter, Score, Facet — are formally defined as functions producing posting lists. Their compositions form a monoid under function composition (Theorem 3.2.3), and derived operators for hybrid queries (HybridTextVector, SemanticFilter) are constructed from these primitives.
- **Join operations.** Cross-paradigm joins — TextVectorJoin, VectorRelationalJoin, and a generalized multi-paradigm join — are formally defined with proofs of commutativity (up to payload ordering) and associativity (Theorems 4.3.1, 4.3.2). Generalized posting lists carry multi-document tuples for join results.
- **Hierarchical data.** Path expressions and unnest operators handle nested JSON-like documents within the posting list framework. All hierarchical operators (PathFilterOperator, PathUnnestOperator, PathAggregateOperator, VertexAggregateOperator) implement `cost_estimate()` for integration with the query

optimizer's cost model.

- **Category theory.** A query category with posting lists as objects and operators as morphisms is established. Functors between paradigm-specific subcategories (relational, text, vector) formalize the cross-paradigm transformations. Natural transformations enable systematic rewriting rules for query optimization.
- **Query optimization.** Lattice-based rewriting rules — filter pushdown, vector threshold merge, intersect reordering by cardinality — are proven to preserve semantic equivalence while reducing computational cost.

2.2 Paper 2: Extending the Unified Mathematical Framework to Support Graph Data Structures (December 2024)

This paper extends the framework to incorporate graph data — vertices, edges, traversals, and pattern matching — as first-class citizens.

Key results:

- **Graph posting lists.** A graph posting list carries (id, subgraph) pairs. A bijective mapping between graph posting lists and standard posting lists establishes an isomorphism that preserves all Boolean algebra operations (Theorem 1.1.6).
- **Graph operators.** Three fundamental graph operators are defined: **Traverse** (BFS from a starting vertex with label filtering and hop limit), **PatternMatch** (subgraph isomorphism with vertex/edge constraints, optimized with candidate pre-computation, arc consistency pruning, MRV variable ordering, and incremental edge validation), and **Regular Path Query** (NFA-based evaluation of path expressions like `cited_by/cited_by`).
- **Cross-paradigm integration.** Operators for converting between graphs and relational data (ToGraph, FromGraph), graph-vector integration (vertex embeddings, vector-enhanced pattern matching), and graph-text integration (semantic graph search) are formally defined.
- **Complexity analysis.** Pattern matching is NP-complete in the worst case ($O(|V_G|^{|V_P|})$); regular path queries are polynomial ($O(|V|^2 \cdot |R|)$); traversal is $O(\sum_{i=1}^{k} d^i)$ where d is the average degree.

2.3 Paper 3: Bayesian BM25 — A Probabilistic Framework for Hybrid Text and Vector Search (January 2026)

This paper addresses a five-decade gap in information retrieval: BM25 scores, despite being derived from a probabilistic model, are not probabilities. They are unbounded, query-dependent, and incompatible with other ranking signals. Bayesian BM25 closes this gap.

The problem. Standard BM25 scores range over $[0, +\infty)$. Directly combining them with bounded signals like cosine similarity ($[0, 1]$) causes signal dominance (Theorem 1.2.2). Reciprocal Rank Fusion (RRF) discards score magnitudes, losing confidence information (Theorem 1.3.2).

The solution. Bayesian BM25 applies Bayes' theorem with a **sigmoid likelihood model** ($P(s \mid R{=}1) = \sigma(\alpha(s - \beta))$), a **composite prior** incorporating term frequency and document length signals (Definition 4.2.3), and a **corpus-level base rate prior** capturing the global prevalence of relevance (Definition 4.4.1).

Key results:

- **Three-term posterior decomposition.** The log-odds of the posterior decomposes additively into likelihood, base rate, and document prior contributions (Theorem 4.4.2), enabling efficient computation via two successive Bayes updates.
- **Monotonicity preservation.** The Bayesian transformation preserves the ranking order of BM25 for fixed prior (Theorem 4.3.1), ensuring that higher BM25 scores still indicate higher relevance probability.
- **Calibration.** Expected calibration error is reduced by 68-77% compared to raw sigmoid calibration, without requiring relevance labels (Section 4.5).
- **WAND/BMW compatibility.** The transformation preserves safe document pruning through modified upper bounds that account for document-dependent priors (Theorem 6.1.2). Block-Max WAND partitions posting lists into fixed-size blocks and precomputes per-block maximum scores, enabling safe skipping of entire blocks during top-k retrieval.
- **Hybrid search fusion.** Calibrated probability signals from BM25 and vector similarity can be combined through probabilistic AND (product rule) and OR (inclusion-exclusion) operations. Log-space computation ensures numerical stability for extreme probabilities (Section 5.5).
- **Progressive parameter estimation.** The sigmoid parameters α and β can be estimated online from the score distribution without relevance labels, using median and IQR heuristics (Section 8.3).

2.4 Paper 4: From Bayesian Inference to Neural Computation (February 2026)

This paper demonstrates that feedforward neural network structure **emerges analytically** from first-principles Bayesian inference over multiple relevance signals. Starting from the question "what is the probability that a document is relevant given multiple evidence signals?", the derivation arrives at a neural network without designing one.

The derivation proceeds in three stages:

1. **Calibration.** Each scoring signal produces a calibrated probability $P_i \in (0, 1)$.
2. **Log-odds aggregation.** Probabilities are mapped to log-odds via the logit function and combined as a weighted sum (normalized Logarithmic Opinion Pooling).
3. **Posterior computation.** The scaled mean log-odds passes through a sigmoid to produce the final probability.

Key results:

- **Neural structure as theorem.** The end-to-end computation has the structure of a feedforward neural network: inputs are calibrated to probabilities, pass through a logit nonlinearity (hidden layer), undergo linear aggregation with confidence scaling, and pass through a sigmoid output activation (Theorem 5.2.1). When all signals share sigmoid calibration, the logit-sigmoid identity collapses the hidden layer, yielding logistic regression (Theorem 5.2.1a). When signals have heterogeneous calibrations (the hybrid search case with BM25 + vector similarity), the logit performs a genuine nonlinear transformation, producing a true two-layer network.
- **Activation functions as probabilistic answers.** The sigmoid answers "how probable?" (Bayesian posterior for binary relevance). ReLU answers "how much?" (MAP estimator under sparse non-negative priors, Theorem 6.5.3). Swish answers "what is the expected

relevant amount?" (Bayesian expected value under the same sparse gating structure, Theorem 6.7.4). GELU is the Gaussian approximation of Swish (Theorem 6.8.1). The ReLU-to-Swish transition is the MAP-to-Bayes duality of classical statistics applied to neural activations.

- **Log-odds conjunction resolves shrinkage.** Naive probabilistic conjunction (product rule) causes scores to shrink toward zero as more signals agree — violating the intuition that agreement should increase confidence. The log-odds mean with multiplicative confidence scaling ($\sqrt{n}$ law) preserves scale neutrality and sign: agreement among irrelevant signals cannot produce a relevance judgment (Corollary 4.2.3), a structural guarantee absent from additive formulations.
- **Attention as Logarithmic Opinion Pooling.** Relaxing the uniform reliability assumption — allowing weights to depend on query-signal interaction — yields the attention mechanism, mathematically equivalent to Hinton's Product of Experts with context-dependent reliability (Theorem 8.3). This provides a probabilistic justification for why attention computes a weighted sum.
- **Exact neural pruning.** WAND and Block-Max WAND algorithms from information retrieval constitute provably exact pruning methods for sigmoid-bounded neural structures, with formal safety guarantees unattainable with ReLU's unboundedness.
- **Depth from recursive inference.** Deep networks are chains of iterated marginalization over latent variables (Theorem 9.1.1). Each layer constructs the evidence required by the next.

2.5 Paper 5: Vector Scores as Likelihood Ratios — Index-Derived Bayesian Calibration for Hybrid Search (March 2026)

This paper addresses a fundamental gap in hybrid search: vector similarity scores (cosine similarity, inner product, Euclidean distance) are geometric quantities, not probabilities. A cosine similarity of 0.85 does not mean an 85% chance of relevance, yet hybrid systems routinely combine such scores with calibrated lexical signals through ad-hoc normalization (min-max, arctangent) or rank-based fusion (RRF) that discards score magnitude information entirely.

The problem. Any fixed monotonic transformation $g : \mathbb{R} \to [0, 1]$ applied to vector scores fails to account for the local density structure of the embedding space (Theorem 1.2.2). Two documents equidistant from a query carry different relevance information depending on whether they are in a dense cluster (many nearby documents) or a sparse region (few nearby documents). No query-independent function can capture this distinction.

The solution. A Bayesian calibration framework transforms vector scores into calibrated relevance probabilities through a likelihood ratio formulation:

$$\text{logit } P(R = 1 \mid d) = \log \frac{f_R(d)}{f_G(d)} + \text{logit } P(R = 1) \tag{1}$$

where $f_R(d)$ is the local distance density among relevant documents and $f_G(d)$ is the global background density.

Key results:

- **Structural identity with BM25.** The log-odds posterior for vector calibration has the same additive structure as Bayesian BM25 (Remark 3.1.2). Both paradigms are calibrated through the same Bayesian likelihood ratio structure, each drawing on the statistics of its

native index.

- **Index-derived statistics.** Both densities are extracted from statistics already computed during ANN index construction and search — IVF cell populations and intra-cluster distances, HNSW edge distances and search trajectories — at negligible additional cost (Section 6).
- **Circularity resolution.** The fundamental circularity problem (estimating the local density requires knowing which documents are relevant) is resolved through cross-modal conditional independence: any relevance signal conditionally independent of vector distance given true relevance (e.g., lexical matching, alternative embedding models) provides importance weights that break the self-referential loop (Section 4).
- **Unified hybrid fusion.** Calibrated vector scores integrate with Bayesian BM25 through additive log-odds, completing the probabilistic unification of sparse and dense retrieval where every signal contributes independently calibrated Bayesian evidence.

3. Core Concepts

3.1 The Posting List

The posting list is the central abstraction in UQA. Every query operation — regardless of paradigm — produces a posting list.

A posting list is an ordered sequence of (doc_id, payload) pairs, sorted by doc_id. Each payload contains:

- **positions**: term positions within the document (for text retrieval)
- **score**: a relevance score (float)
- **fields**: a dictionary of metadata

The ordering by doc_id enables efficient two-pointer merge algorithms for Boolean operations:

- **Union** (OR): merge two sorted lists, keeping all entries
- **Intersection** (AND): merge two sorted lists, keeping only shared doc_ids
- **Complement** (NOT): subtract one list from the universal set

These operations form a complete Boolean algebra, meaning any Boolean combination of any paradigm's results is well-defined and efficiently computable.

3.2 Operators

Operators are functions that produce posting lists. They are the building blocks of query plans.

Primitive operators:

Operator	Input	Output	Description
TermOperator	term string	PostingList	Documents containing the term
KNNOperator	query vector, k	PostingList	k nearest neighbors by cosine similarity
VectorSimilarityOperator	query vector, θ	PostingList	Documents with similarity $\geq \theta$
SpatialWithinOperator	field, center, radius	PostingList	Points within radius (R*Tree + Haversine)
FilterOperator	field, predicate	PostingList	Documents matching the predicate
ScoreOperator	source, scorer	PostingList	Adds relevance scores to documents
FacetOperator	field	dict	Value counts for a field

Boolean operators:

Operator	Semantics
UnionOperator	$A_1 \cup A_2 \cup \cdots \cup A_n$
IntersectOperator	$A_1 \cap A_2 \cap \cdots \cap A_n$
ComplementOperator	$\overline{A}$

Fusion operators:

Operator	Semantics
LogOddsFusionOperator	Combine calibrated probabilities via log-odds mean
ProbBoolFusionOperator	Probabilistic AND or OR
ProbNotOperator	Probabilistic NOT: $P' = 1 - P$

Advanced fusion operators:

Operator	Semantics
AttentionFusionOperator	Attention-weighted log-odds conjunction using query features
LearnedFusionOperator	Learned per-signal weight log-odds conjunction

Scoring operators:

Operator	Semantics
SparseThresholdOperator	ReLU thresholding: max(0, score - threshold)
MultiFieldSearchOperator	Per-field Bayesian BM25 with cross-field log-odds fusion

Pipeline operators:

Operator	Semantics
MultiStageOperator	Cascading retrieval with per-stage top-k or threshold cutoffs

Graph operators:

Operator	Semantics
TraverseOperator	BFS traversal from a vertex (configurable `score`, default 0.9)
PatternMatchOperator	Subgraph isomorphism matching (MRV + arc consistency + incremental edge validation; configurable `score`, default 0.9)
RegularPathQueryOperator	NFA-based regular path evaluation (configurable `score`, default 0.9)

Temporal graph operators:

Operator	Semantics
TemporalTraverseOperator	BFS traversal filtered by valid_from/valid_to edge properties (configurable `score`, default 0.9)
TemporalPatternMatchOperator	Subgraph isomorphism with temporal edge constraints (configurable `score`, default 0.9)

GNN operators:

Operator	Semantics
MessagePassingOperator	K-layer neighbor feature aggregation with sigmoid calibration
GraphEmbeddingOperator	Structural embeddings from degree, labels, and k-hop connectivity

Deep learning operators:

Operator	Semantics
DeepFusionOperator	Multi-layer Bayesian fusion (conv, pool, dense, softmax, attention) over graph neighborhoods
DeepLearnOperator	Analytical CNN training via PoE local learning with supervised conv weight estimation, attention, and pruning
TrainedModel	Persisted representation of a trained model (weights, specs, expert heads, pruning metadata)

Graph maintenance:

Operator	Semantics
GraphDelta	Records add/remove vertex/edge operations for atomic application
VersionedGraphStore	Version-tracked graph with apply/rollback and inverse delta storage

All graph operators accept an optional `score` parameter (default: `DEFAULT_GRAPH_SCORE = 0.9`) that controls the score assigned to reachable/matched vertices. This allows tuning the graph contribution to log-odds fusion without modifying the operator source.

All operators implement an `execute(context)` method that returns a PostingList, and a `cost_estimate(stats)` method used by the query optimizer.

3.3 Scoring

UQA supports multiple scoring models, all of which produce values in $[0, 1]$:

BM25 (traditional lexical scoring):

$$\text{score}(f, n) = w - \frac{w}{1 + f \cdot \text{inv_norm}} \tag{2}$$

where $w = \text{boost} \cdot \text{IDF}(t)$ and inv_norm encodes document length normalization. Parameters: $k_1 = 1.2$, $b = 0.75$.

Bayesian BM25 (calibrated probabilistic scoring): Transforms BM25 raw scores into $P(\text{relevant}) \in [0, 1]$ via Bayesian inference with a sigmoid likelihood model. The posterior decomposes into three additive log-odds terms: likelihood, base rate, and document prior.

Vector similarity (semantic scoring): Cosine similarity in $[-1, 1]$ is linearly mapped to probability space: $P = (1 + \text{sim})/2$.

External prior scoring: Combines BM25 likelihood with document-level prior features (recency, authority) via log-odds addition. The prior maps document metadata to a probability in $(0, 1)$ through user-supplied functions (`recency_prior`, `authority_prior`).

Multi-field scoring: Scores each text field independently with Bayesian BM25, then fuses per-field posteriors via weighted log-odds conjunction. Each field can have independent calibration parameters (alpha, beta, base_rate).

Calibration diagnostics: Expected Calibration Error (ECE), Brier score, and reliability diagrams measure how well predicted probabilities match actual relevance rates. `ParameterLearner` fits calibration parameters from relevance judgments via batch training or online updates.

3.4 Multi-Signal Fusion

When combining signals from different paradigms, UQA offers two approaches:

Log-odds conjunction (recommended): Maps each calibrated probability to log-odds via the logit function, computes the arithmetic mean, scales by n^α (default $\alpha = 0.5$ for $\sqrt{n}$ confidence scaling), and applies sigmoid to return to probability space. This resolves the conjunction shrinkage problem: agreement among signals amplifies confidence without penalizing for signal count.

$$P_{\text{final}} = \sigma\left(\frac{1}{n^{1-\alpha}} \sum_{i=1}^{n} \text{logit}(P_i)\right) \tag{3}$$

Properties:

- Scale neutrality: if all $P_i = p$, then $P_{\text{final}} = p$ regardless of n
- Sign preservation: agreement among irrelevant signals cannot produce relevance
- Irrelevance preservation: all $P_i < 0.5$ implies $P_{\text{final}} < 0.5$
- Relevance preservation: all $P_i > 0.5$ implies $P_{\text{final}} > 0.5$

Probabilistic Boolean: Standard probability rules under independence.

- AND: $P = \prod_i P_i$
- OR: $P = 1 - \prod_i (1 - P_i)$
- NOT: $P = 1 - P_{\text{signal}}$

Attention-weighted fusion: Uses query-level features (IDF statistics, coverage, query length) to compute per-signal attention weights via softmax over a learned weight matrix. Delegates to `AttentionLogOddsWeights` from the `bayesian_bm25` package. Supports batch training via `.fit()` and online updates via `.update()`, with `state_dict()` / `load_state_dict()` for persistence.

Learned-weight fusion: Uses per-signal weights learned from relevance judgments without query features. Delegates to `LearnableLogOddsWeights` from the `bayesian_bm25` package.

Multi-stage retrieval: Cascading pipeline where each stage refines the candidate set from the previous stage. Each stage is an (operator, cutoff) pair where cutoff is either top-k (integer) or a score threshold (float). Stage 0 executes fully; subsequent stages re-score surviving documents.

4. Function Reference

4.1 WHERE-Clause Functions

`text_match(field, query)`

Full-text search with BM25 scoring. Tokenizes the query string, retrieves the posting list for each token from the inverted index, intersects them, and scores results using BM25.

```
SELECT title, _score FROM papers
WHERE text_match(title, 'attention transformer')
ORDER BY _score DESC;
```

Parameter	Type	Description
`field`	column name	The text column to search
`query`	string	Space-separated query terms

Returns: PostingList with `_score` set to the BM25 score (≥ 0, unbounded).

`bayesian_match(field, query)`

Bayesian BM25 search. Same tokenization and retrieval as `text_match`, but applies the Bayesian calibration transform so that `_score` is $P(\text{relevant}) \in [0, 1]$.

```sql
SELECT title, _score FROM papers
WHERE bayesian_match(title, 'attention transformer')
ORDER BY _score DESC;
```

Parameter	Type	Description
`field`	column name	The text column to search
`query`	string	Space-separated query terms

Returns: PostingList with `_score` as a calibrated probability in $[0, 1]$.

`knn_match(field, vector, k)`

K-nearest neighbor vector search. Uses the IVF index if one exists on the column (created via `CREATE INDEX ... USING hnsw` or `USING ivf`), otherwise falls back to brute-force exact cosine similarity scan. The vector argument accepts either an `ARRAY[...]` literal or a `$N` parameter reference.

```python
# Option A: parameter binding
qv = np.random.randn(64).astype(np.float32)
result = engine.sql(
    "SELECT title, _score FROM papers WHERE knn_match(embedding, $1, 10) ORDER BY _score DESC",
    params=[qv],
)

# Option B: inline ARRAY literal
result = engine.sql(
    "SELECT title, _score FROM papers WHERE knn_match(embedding, ARRAY[0.1, 0.2, ...], 10) ORDER BY _score DESC"
)
```

Parameter	Type	Description
`field`	column name	The VECTOR column to search
`vector`	ARRAY or $N	Query vector (float32)
`k`	integer	Number of nearest neighbors to retrieve

Returns: PostingList with `_score` set to cosine similarity.

`traverse_match(start, label, max_hops)`

Graph reachability as a scored signal. Performs BFS from a starting vertex along edges with the given label, up to `max_hops` hops. Reachable vertices receive a score of 0.9. The FROM-clause variant `traverse()` (Section 4.4) supports named graphs via a direct graph name argument (the `graph:` prefix is accepted for backward compatibility but not required).

```sql
SELECT title, _score FROM papers
WHERE traverse_match(1, 'cited_by', 2)
ORDER BY _score DESC;
```

Parameter	Type	Description
`start`	integer	Starting vertex ID
`label`	string	Edge label to follow
`max_hops`	integer	Maximum traversal depth (default: 1)

Returns: PostingList of reachable vertices with `_score = 0.9`.

`path_filter(path, value)` / `path_filter(path, op, value)`

Filter on nested (hierarchical) document fields. The 2-argument form tests equality; the 3-argument form applies a comparison operator.

```sql
-- Equality
SELECT * FROM orders WHERE path_filter('shipping.city', 'Seoul');

-- Comparison
SELECT * FROM orders WHERE path_filter('items.price', '>', 100);
```

Parameter	Type	Description
`path`	string	Dot-separated path into the nested document
`op`	string	Comparison operator: `=`, `!=`, `<`, `>`, `<=`, `>=`
`value`	any	Value to compare against

If any element in a nested array matches, the document is included.

`spatial_within(field, POINT(x, y), distance_meters)`

Geospatial range query. Returns all points within `distance_meters` of the center point, scored by proximity: `score = 1.0 - (dist / distance_meters)`. Uses the R*Tree spatial index if one exists on the column, otherwise falls back to brute-force Haversine scan.

```sql
-- All restaurants within 2km of Washington Square Park
SELECT name, cuisine, rating FROM restaurants
WHERE spatial_within(location, POINT(-73.9973, 40.7308), 2000)
ORDER BY rating DESC;

-- With parameter binding
SELECT name FROM restaurants
WHERE spatial_within(location, $1, $2) ORDER BY name;
-- params: [[-73.9973, 40.7308], 2000]
```

Parameter	Type	Description
`field`	column name	The POINT column to search
`POINT(x, y)`	POINT or $N	Center point (longitude, latitude)
`distance_meters`	numeric or $N	Search radius in meters

Returns: PostingList with `_score` proportional to proximity.

`vector_exclude(field, positive_vector, negative_vector, k, threshold)`

KNN search with negative vector exclusion. Retrieves k nearest neighbors to the positive vector, then removes documents whose cosine similarity to the negative vector exceeds the threshold.

```
result = engine.sql(
    "SELECT title, _score FROM papers "
    "WHERE vector_exclude(embedding, $1, $2, 10, 0.8) ORDER BY _score DESC",
    params=[query_vec, negative_vec],
)
```

Parameter	Type	Description
`field`	column name	The VECTOR column to search
`positive_vector`	ARRAY or $N	Query vector for KNN retrieval
`negative_vector`	ARRAY or $N	Negative vector for exclusion
`k`	integer	Number of nearest neighbors to retrieve
`threshold`	float	Exclusion threshold for negative similarity

`sparse_threshold(signal, threshold)`

ReLU thresholding operator (Paper 4, Section 6.5). Applies `max(0, score - threshold)` to each document score, excluding documents whose adjusted score is zero. Implements the MAP estimation interpretation of ReLU activation.

```
SELECT title, _score FROM papers
WHERE sparse_threshold(bayesian_match(title, 'attention'), 0.3)
ORDER BY _score DESC;
```

Parameter	Type	Description
`signal`	signal function	A signal function (bayesian_match, text_match, etc.)
`threshold`	float	Score threshold; documents below this are excluded

Returns: PostingList with adjusted scores `max(0, score - threshold)`.

`multi_field_match(field1, field2, ..., query[, w1, w2, ...])`

Multi-field Bayesian BM25 search (Paper 3, Section 12.2 #1). Scores each field independently with Bayesian BM25, then fuses per-field posteriors via weighted log-odds conjunction. Optional trailing numeric arguments set per-field weights (normalized to sum to 1).

```sql
-- Equal-weight search across title and abstract
SELECT title, _score FROM papers
WHERE multi_field_match(title, abstract, 'attention transformer')
ORDER BY _score DESC;

-- Title weighted 2x higher than abstract
SELECT title, _score FROM papers
WHERE multi_field_match(title, abstract, 'attention', 2.0, 1.0)
ORDER BY _score DESC;
```

Parameter	Type	Description
`field1, field2, ...`	column names	Text columns to search (at least 2)
`query`	string	Search query
`w1, w2, ...`	floats (optional)	Per-field weights (default: equal)

Returns: PostingList with fused calibrated probabilities in $[0, 1]$.

`bayesian_match_with_prior(field, query, prior_field, prior_mode)`

Bayesian BM25 with external document-level prior (Paper 3, Section 12.2 #6). Combines BM25 likelihood with a prior computed from document metadata via log-odds addition: `logit(posterior) = logit(likelihood) + logit(prior)`.

```sql
SELECT title, _score FROM papers
WHERE bayesian_match_with_prior(title, 'attention', 'authority', 'authority')
ORDER BY _score DESC;
```

Parameter	Type	Description
`field`	column name	Text column to search
`query`	string	Search query
`prior_field`	string	Document field for prior computation
`prior_mode`	string	`'recency'` (exponential decay) or `'authority'` (categorical levels)

Returns: PostingList with posterior probabilities incorporating the external prior.

`temporal_traverse(start, label, hops, timestamp[, source])` / `temporal_traverse(start, label, hops, from_ts, to_ts[, source])`

Time-aware graph traversal (Paper 2, Section 10). Same BFS as `traverse_match` but filters edges by `valid_from` / `valid_to` temporal properties. The 4-argument form checks a single timestamp; the 5-argument form checks interval overlap. An optional trailing graph name argument targets a named graph instead of the per-table graph store. The `graph:` prefix is accepted for backward compatibility but not required.

```
-- Point-in-time traversal
SELECT * FROM temporal_traverse(1, 'knows', 2, 1700000000);

-- Range traversal
SELECT * FROM temporal_traverse(1, 'knows', 2, 1690000000, 1710000000);

-- Named graph traversal (direct name)
SELECT * FROM temporal_traverse(1, 'knows', 2, 150, 'net');
```

Parameter	Type	Description
`start`	integer	Starting vertex ID
`label`	string	Edge label to follow
`hops`	integer	Maximum traversal depth
`timestamp`	float	Point-in-time filter
`from_ts, to_ts`	floats	Time range filter (interval overlap)
`source`	string	Optional graph name to target a named graph

Returns: PostingList of reachable vertices with `_score = 0.9`.

`message_passing(k_layers, aggregation, property_name)`

GNN-style k-layer message-passing aggregation (Paper 2 + Paper 4). Each vertex accumulates neighbor feature values over k rounds, then results are calibrated via sigmoid.

```
SELECT * FROM papers
WHERE message_passing(2, 'mean', 'score');
```

Parameter	Type	Description
`k_layers`	integer	Number of aggregation layers (default: 2)
`aggregation`	string	Aggregation function: `'mean'`, `'sum'`, or `'max'`
`property_name`	string	Vertex property to aggregate (optional)

Returns: PostingList of all vertices with sigmoid-calibrated aggregated scores.

`graph_embedding(dimensions, k_layers)`

Structural graph embedding computation (Paper 2 + Paper 4). Computes per-vertex embeddings from structural features: degree (in/out), edge label distribution, and k-hop connectivity statistics. Embeddings are L2-normalized and stored in the `_embedding` payload field.

```
SELECT * FROM papers WHERE graph_embedding(16, 2);
```

Parameter	Type	Description
`dimensions`	integer	Embedding dimensionality (default: 32)
`k_layers`	integer	Hop depth for connectivity features (default: 2)

Returns: PostingList with `_embedding` field containing the structural embedding vector.

4.2 Fusion Meta-Functions

Fusion functions combine multiple signal functions into a single relevance score. Each signal argument must be a signal function call (`text_match`, `bayesian_match`, `bayesian_match_with_prior`, `knn_match`, `bayesian_knn_match`, `traverse_match`, `spatial_within`, `pagerank`, `hits`, `betweenness`, `weighted_rpq`, or `message_passing`). Inside a fusion context, `text_match` is automatically promoted to Bayesian BM25 calibration so all signals produce probabilities in $(0, 1)$.

`fuse_log_odds(signal1, signal2, ...[, alpha])`

Log-odds conjunction. Maps each signal's calibrated probability to log-odds, computes the mean, scales by n^α, and applies sigmoid. The optional last numeric argument sets α (default: 0.5).

```
SELECT title, _score FROM papers
WHERE fuse_log_odds(
    text_match(title, 'attention'),
    knn_match(embedding, ARRAY[0.1, 0.2, ...], 5),
    traverse_match(1, 'cited_by', 2)
) ORDER BY _score DESC;

-- Custom alpha
SELECT title, _score FROM papers
WHERE fuse_log_odds(
    bayesian_match(title, 'neural'),
    knn_match(embedding, ARRAY[0.1, 0.2, ...], 10),
    0.7
) ORDER BY _score DESC;
```

`fuse_mean(signal1, signal2, ...)`

Log-odds mean aggregation (Definition 4.1.1, Paper 4). Computes the arithmetic mean in log-odds space and maps back to probability via sigmoid. Unlike `fuse_log_odds`, no confidence scaling (n^α) is applied -- the result is purely scale-neutral: if all signals report the same probability p, the output is exactly p regardless of n. This is the normalized Logarithmic Opinion Pool (Theorem 4.1.2a).

```sql
SELECT title, _score FROM papers
WHERE fuse_mean(
    bayesian_match(title, 'attention'),
    knn_match(embedding, ARRAY[0.1, 0.2, ...], 5)
) ORDER BY _score DESC;
```

`fuse_prob_and(signal1, signal2, ...)`

Probabilistic AND. Computes $P = \prod_i P_i$.

```sql
SELECT title, _score FROM papers
WHERE fuse_prob_and(
    text_match(title, 'attention'),
    knn_match(embedding, ARRAY[0.1, 0.2, ...], 5)
) ORDER BY _score DESC;
```

`fuse_prob_or(signal1, signal2, ...)`

Probabilistic OR. Computes $P = 1 - \prod_i (1 - P_i)$.

```sql
SELECT title, _score FROM papers
WHERE fuse_prob_or(
    text_match(title, 'attention'),
    knn_match(embedding, ARRAY[0.1, 0.2, ...], 5)
) ORDER BY _score DESC;
```

`fuse_prob_not(signal)`

Probabilistic NOT. Computes $P = 1 - P_{\text{signal}}$. Takes exactly one signal argument.

```sql
SELECT title, _score FROM papers
WHERE fuse_prob_not(text_match(title, 'noise'))
ORDER BY _score DESC;
```

`fuse_attention(signal1, signal2, ...)`

Attention-weighted log-odds fusion (Paper 4, Section 8). Computes per-signal attention weights from query-level features (IDF statistics, coverage, query length) via a learned weight matrix, then fuses via weighted log-odds conjunction.

```sql
SELECT title, _score FROM papers
WHERE fuse_attention(
    bayesian_match(title, 'attention'),
    bayesian_match(abstract, 'transformer')
) ORDER BY _score DESC;
```

`fuse_learned(signal1, signal2, ...)`

Learned-weight log-odds fusion (Paper 4, Section 8). Uses per-signal weights learned from relevance judgments (no query features needed).

```sql
SELECT title, _score FROM papers
WHERE fuse_learned(
    bayesian_match(title, 'attention'),
    knn_match(embedding, ARRAY[0.1, 0.2, ...], 5)
) ORDER BY _score DESC;
```

`staged_retrieval(signal1, k1, signal2, k2, ...)`

Multi-stage cascading retrieval pipeline (Paper 4, Section 9). Each (signal, cutoff) pair defines a stage. Stage 0 executes fully and retains the top-k (integer cutoff) or above-threshold (float cutoff) documents. Subsequent stages re-score the surviving candidates.

```sql
-- Two-stage: broad recall (top 50) then precise re-ranking (top 10)
SELECT title, _score FROM papers
WHERE staged_retrieval(
    bayesian_match(title, 'transformer'), 50,
    bayesian_match(abstract, 'self attention mechanism'), 10
) ORDER BY _score DESC;
```

4.3 SELECT Scalar Functions

`path_agg(path, func)`

Per-row aggregation over nested arrays. Navigates the dot-separated path into each document and applies the aggregation function to the array of values found.

```sql
SELECT path_agg('items.price', 'sum') AS total,
       path_agg('items.quantity', 'count') AS num_items
FROM orders
WHERE path_filter('shipping.city', 'Seoul');
```

Parameter	Type	Description
`path`	string	Dot-separated path to a nested array field
`func`	string	Aggregation function: `sum`, `count`, `avg`, `min`, `max`, `quantile`

Aggregation monoids: Each aggregation function is backed by a monoid (identity + associative combine) that enables parallel decomposition (Theorem 5.1.5, Paper 1): CountMonoid, SumMonoid, AvgMonoid (sum/count pair), MinMonoid, MaxMonoid, and **QuantileMonoid**. QuantileMonoid collects values and computes the requested quantile at finalize using linear interpolation between adjacent sorted values. `quantile=0.5` gives the median, `quantile=0.95` gives P95, etc.

`path_value(path)`

Access a nested field value by dot-path.

```sql
SELECT path_value('shipping.city') AS city,
       path_value('customer.name') AS name
FROM orders;
```

Parameter	Type	Description
`path`	string	Dot-separated path to a nested field

`uqa_highlight(field, query [, start_tag, end_tag [, max_fragments, fragment_size]])`

Highlight matched query terms in the field text. Returns the text with matched tokens wrapped in tags. Uses the table's analyzer for stemming-aware matching.

```
-- Default <b>...</b> tags
SELECT title, uqa_highlight(body, 'database query') AS snippet
FROM articles WHERE body @@ 'database query';

-- Custom tags
SELECT title, uqa_highlight(body, 'search', '<em>', '</em>') AS snippet
FROM articles WHERE body @@ 'search';

-- Snippet extraction: 2 fragments of 100 chars each
SELECT title, uqa_highlight(body, 'query', '<b>', '</b>', 2, 100) AS snippet
FROM articles WHERE body @@ 'query';
```

Parameter	Type	Default	Description
`field`	column	(required)	Text column to highlight
`query`	string	(required)	FTS query string (same syntax as `@@`)
`start_tag`	string	`'<b>'`	Opening markup tag
`end_tag`	string	`'</b>'`	Closing markup tag
`max_fragments`	integer	`0`	Number of fragments to extract (0 = full text)
`fragment_size`	integer	`150`	Maximum characters per fragment

The query string is parsed using the same FTS grammar as the `@@` operator. Boolean operators (`AND`, `OR`, `NOT`), quoted phrases, and field-prefixed terms are all supported. The highlighter analyzes both the text tokens and query terms with the table's analyzer, so stemmed queries correctly highlight inflected forms.

`uqa_facets(field1 [, field2, ...])`

Compute facet counts over the search results. Replaces normal row output with summary rows showing distinct values and their counts.

```
-- Single field: returns facet_value | facet_count
SELECT uqa_facets(category) FROM articles WHERE body @@ 'database';

-- Multi-field: returns facet_field | facet_value | facet_count
SELECT uqa_facets(category, author) FROM articles WHERE body @@ 'database';
```

Parameter	Type	Description
`field1`	column	First field to facet on
`field2, ...`	column	Additional fields (optional)

Single-field output: `facet_value | facet_count`. Multi-field output: `facet_field | facet_value | facet_count`. Values are sorted alphabetically. Facet counts respect all WHERE-clause predicates.

`POINT(x, y)`

Construct a point value from longitude and latitude.

```
SELECT POINT(-73.9857, 40.7484) AS empire_state;
INSERT INTO restaurants (name, location) VALUES ('Joes Pizza',
POINT(-73.9969, 40.7306));
```

`ST_Distance(point1, point2)`

Compute the Haversine great-circle distance between two POINT values, in meters.

```
SELECT name,
       ROUND(ST_Distance(location, POINT(-73.9857, 40.7484)), 0) AS dist_m
FROM restaurants
WHERE spatial_within(location, POINT(-73.9857, 40.7484), 5000)
ORDER BY dist_m;
```

`ST_Within(point1, point2, distance_meters)` / `ST_DWithin(...)`

Boolean distance predicate. Returns true if the Haversine distance between the two points is less than or equal to `distance_meters`. `ST_DWithin` is an alias.

```
SELECT name, cuisine FROM restaurants
WHERE ST_DWithin(location, POINT(-73.9903, 40.7359), 1500)
ORDER BY name;
```

4.4 FROM-Clause Table Functions

`traverse(start, label, max_hops[, source])`

BFS graph traversal as a virtual table. Returns one row per reachable vertex with its properties as columns. An optional trailing graph name argument targets a named graph (created via `create_graph()` or `cypher()`) instead of the per-table graph store. The `graph:` prefix is accepted for backward compatibility but not required.

```sql
SELECT _doc_id, _score FROM traverse(1, 'cited_by', 2);

-- With WHERE filtering on vertex properties
SELECT _doc_id, title FROM traverse(1, 'cited_by', 3)
WHERE _score > 0.5;

-- Named graph traversal (direct name)
SELECT * FROM traverse(1, 'knows', 2, 'social');
```

Parameter	Type	Description
`start`	integer	Starting vertex ID
`label`	string	Edge label to follow
`max_hops`	integer	Maximum traversal depth
`source`	string	Optional graph name to target a named graph

`rpq(path_expr, start[, source])`

Regular path query. Evaluates a regular path expression using NFA simulation and returns matching vertices. An optional trailing graph name argument targets a named graph instead of the per-table graph store. The `graph:` prefix is accepted for backward compatibility but not required.

```sql
-- Follow two 'cited_by' edges
SELECT _doc_id, title FROM rpq('cited_by/cited_by', 1);

-- Alternation: follow 'cited_by' or 'references'
SELECT _doc_id FROM rpq('cited_by|references', 1);

-- Named graph RPQ (direct name)
SELECT * FROM rpq('knows/works_with', 1, 'social');
```

Parameter	Type	Description
`path_expr`	string	Path expression using `/` (concatenation) and `|` (alternation)
`start`	integer	Starting vertex ID (optional)
`source`	string	Optional graph name to target a named graph

`temporal_traverse(start, label, hops, timestamp[, source])` / `temporal_traverse(start, label, hops, from_ts, to_ts[, source])`

Time-aware graph traversal as a FROM-clause table function. Same BFS as `traverse()` but filters edges by `valid_from`/`valid_to` temporal properties. The 4-argument form checks a single timestamp; the 5-argument form checks interval overlap. An optional trailing graph name argument targets a named graph. The `graph:` prefix is accepted for backward compatibility but not required.

```sql
-- Point-in-time traversal
SELECT * FROM temporal_traverse(1, 'knows', 2, 1700000000);

-- Range traversal with named graph (direct name)
SELECT * FROM temporal_traverse(1, 'knows', 2, 150, 'net');
```

Parameter	Type	Description
`start`	integer	Starting vertex ID
`label`	string	Edge label to follow
`hops`	integer	Maximum traversal depth
`timestamp`	float	Point-in-time filter
`from_ts, to_ts`	floats	Time range filter (interval overlap)
`source`	string	Optional graph name to target a named graph

`text_search(query, field, table)`

Table-scoped full-text search as a FROM-clause source. Returns documents from the specified table matching the query.

```sql
SELECT _doc_id, _score FROM text_search('attention', 'title', 'papers')
ORDER BY _score DESC;
```

Parameter	Type	Description
`query`	string	Search query
`field`	string	Field to search
`table`	string	Table name (optional; defaults to current table context)

`generate_series(start, stop[, step])`

Generates a series of values from `start` to `stop` with an optional `step` increment. Returns rows with a single `generate_series` column.

```sql
SELECT * FROM generate_series(1, 5);
SELECT * FROM generate_series(0, 100, 10);
```

`unnest(array)`

Expands an array to a set of rows. Returns rows with a single `unnest` column.

```sql
SELECT * FROM unnest(ARRAY[1, 2, 3]);
```

`regexp_split_to_table(string, pattern [, flags])`

Splits a string by a regular expression pattern and returns one row per part.

```
SELECT value FROM regexp_split_to_table('one,two,three', ',');
SELECT value FROM regexp_split_to_table('Hello   World', '\s+');
```

Parameter	Type	Description
`string`	string	The string to split
`pattern`	string	Regular expression pattern to split on
`flags`	string	Optional flags: `i` (case-insensitive), `m` (multiline), `s` (dotall)

`json_each(json)` / `json_each_text(json)`

Expands a JSON object into a set of key/value rows. `json_each` returns values as JSON; `json_each_text` returns values as text.

```
SELECT key, value FROM json_each('{"a": 1, "b": 2}');
SELECT key, value FROM json_each_text('{"name": "Alice", "age": "30"}');
```

`json_array_elements(json)` / `json_array_elements_text(json)`

Expands a JSON array into a set of rows. Each row contains a single `value` column.

```
SELECT value FROM json_array_elements('[1, 2, 3]');
SELECT value FROM json_array_elements_text('["a", "b", "c"]');
```

`create_graph(name)` / `drop_graph(name)`

Named graph management functions. `create_graph` creates an isolated graph namespace with dedicated SQLite-backed storage. `drop_graph` removes a named graph and its storage. The aliases `graph_create` and `graph_drop` are also accepted for a consistent `graph_*` naming convention.

```
-- Create a named graph
SELECT * FROM create_graph('social');
SELECT * FROM graph_create('social');   -- alias

-- Drop a named graph (cascades all vertices and edges)
SELECT * FROM drop_graph('social');
SELECT * FROM graph_drop('social');     -- alias
```

Parameter	Type	Description
`name`	string	Graph namespace name

`graph_create_node(graph, label[, properties_json])` / `graph_create_edge(graph, type, src, tgt[, properties_json])`

Standalone property graph node and edge creation. Creates independent vertices and edges in a named graph with auto-generated IDs. Properties are passed as a JSON string. Returns a composite ID in the format `graph:Label:id`.

```
-- Create standalone nodes with JSON properties
SELECT * FROM graph_create_node('social', 'Person', '{"name":"Alice","age":30}');
-- returns: id = 'social:Person:1'

SELECT * FROM graph_create_node('social', 'Person', '{"name":"Bob","age":25}');
-- returns: id = 'social:Person:2'

-- Create edges between nodes
SELECT * FROM graph_create_edge('social', 'KNOWS', 1, 2, '{"since":2020}');
-- returns: id = 'social:KNOWS:1'

-- Properties are optional
SELECT * FROM graph_create_node('social', 'Tag');
SELECT * FROM graph_create_edge('social', 'TAGGED', 1, 3);
```

Parameter	Type	Description
`graph`	string	Named graph (must be created with `create_graph`)
`label` / `type`	string	Vertex label or edge type
`src`	integer	Source vertex ID (edges only)
`tgt`	integer	Target vertex ID (edges only)
`properties_json`	string	Optional JSON object with properties

`graph_nodes(graph[, label][, filter_json])`

Query vertices in a named graph as a virtual table. Optionally filter by label and/or JSON property predicates (exact match on each key-value pair).

```
-- All nodes in a graph
SELECT * FROM graph_nodes('social');

-- Filter by label
SELECT * FROM graph_nodes('social', 'Person');

-- Filter by label and properties
SELECT * FROM graph_nodes('social', 'Person', '{"name":"Alice"}');
```

Column	Type	Description
`id`	integer	Vertex ID
`label`	text	Vertex label
`properties`	text	JSON string of vertex properties

Parameter	Type	Description
`graph`	string	Named graph name
`label`	string	Optional vertex label filter
`filter_json`	string	Optional JSON object for property filtering

`graph_neighbors(graph, id[, type][, direction][, depth])`

Multi-hop BFS neighbor traversal on a named graph. Returns reachable vertices with depth and path information.

```sql
-- 1-hop outgoing neighbors via KNOWS edges
SELECT * FROM graph_neighbors('social', 1, 'KNOWS', 'outgoing', 1);

-- 3-hop traversal in both directions
SELECT * FROM graph_neighbors('social', 1, 'KNOWS', 'both', 3);

-- All edge types, incoming, 2 hops
SELECT * FROM graph_neighbors('social', 3, '', 'incoming', 2);

-- Defaults: outgoing direction, depth 1
SELECT * FROM graph_neighbors('social', 1, 'KNOWS');
```

Column	Type	Description
`id`	integer	Neighbor vertex ID
`label`	text	Vertex label
`properties`	text	JSON string of vertex properties
`depth`	integer	Hop distance from start vertex
`path`	text	JSON array of vertex IDs from start to this vertex

Parameter	Type	Description
`graph`	string	Named graph name
`id`	integer	Start vertex ID
`type`	string	Edge label filter (empty string or omit for all)
`direction`	string	`'outgoing'` (default), `'incoming'`, or `'both'`
`depth`	integer	Maximum traversal hops (default 1)

`graph_traverse(graph, id[, types][, direction][, depth[, strategy]])`

Advanced graph traversal with multiple edge types and BFS/DFS strategy selection. The `types` parameter accepts comma-separated edge labels (e.g., `'CALLS,IMPORTS'`), an `ARRAY` literal (e.g., `ARRAY['CALLS','IMPORTS']`), or `NULL` for all types. The start vertex ID can be a literal, a parameter (`$1`), or a correlated column reference from a LATERAL subquery.

```sql
-- BFS traversal following KNOWS and FOLLOWS edges
SELECT * FROM graph_traverse('social', 1, 'KNOWS,FOLLOWS', 'outgoing', 3, 'bfs');

-- Using ARRAY literal for edge types
SELECT * FROM graph_traverse('social', 1, ARRAY['KNOWS','FOLLOWS'], 'outgoing', 3, 'bfs');

-- NULL means all edge types
SELECT * FROM graph_traverse('social', 1, NULL, 'outgoing', 2);

-- DFS traversal
SELECT * FROM graph_traverse('social', 1, 'CALLS', 'outgoing', 5, 'dfs');

-- All edge types (empty string), defaults to BFS
SELECT * FROM graph_traverse('social', 1, '', 'both', 2);

-- LATERAL: count reachable nodes per start vertex
SELECT s.sid, sub.cnt
FROM start_nodes s,
LATERAL (SELECT COUNT(*) AS cnt
         FROM graph_traverse('social', s.sid, '', 'outgoing', 2, 'bfs'))
sub;
```

Column	Type	Description
`id`	integer	Reached vertex ID
`label`	text	Vertex label
`properties`	text	JSON string of vertex properties
`depth`	integer	Hop distance from start vertex
`path`	text	JSON array of vertex IDs from start to this vertex

Parameter	Type	Description
`graph`	string	Named graph name
`id`	integer	Start vertex ID
`types`	string	Comma-separated edge labels (empty string or omit for all)
`direction`	string	`'outgoing'` (default), `'incoming'`, or `'both'`
`depth`	integer	Maximum traversal hops (default 1)
`strategy`	string	`'bfs'` (default) or `'dfs'`

`graph_edges(graph[, type][, filter_json])` (graph-wide mode)

Query all edges in a named graph as a virtual table. Optionally filter by edge label and/or JSON property predicates.

```
-- All edges in a graph
SELECT * FROM graph_edges('social');

-- Filter by edge type
SELECT * FROM graph_edges('social', 'KNOWS');

-- Count edges
SELECT COUNT(*) FROM graph_edges('social');

-- Filter by type and properties
SELECT * FROM graph_edges('social', 'KNOWS', '{"since":2020}');
```

`graph_edges(graph, vertex_id[, type | NULL][, direction])` (per-vertex mode)

Query edges incident to a specific vertex. When the second argument is an integer, per-vertex mode is activated. Use `NULL` for the type argument to match all edge types. The vertex ID can be a literal, a parameter (`$1`), or a correlated column reference from a LATERAL subquery.

```
-- Outgoing edges from vertex 1
SELECT * FROM graph_edges('social', 1, NULL, 'outgoing');

-- Incoming KNOWS edges to vertex 3
```

```sql
SELECT * FROM graph_edges('social', 3, 'KNOWS', 'incoming');

-- All edges touching vertex 2 (both directions)
SELECT * FROM graph_edges('social', 2, NULL, 'both');

-- LATERAL: count outgoing edges per node
SELECT n.name, sub.cnt
FROM nodes n,
LATERAL (SELECT COUNT(*) AS cnt
         FROM graph_edges('social', n.node_id, NULL, 'outgoing')) sub;

-- LATERAL: list edges per node
SELECT n.name, e.label, e.target_id
FROM nodes n,
LATERAL (SELECT label, target_id
         FROM graph_edges('social', n.node_id, NULL, 'outgoing')) e;
```

Column	Type	Description
`id`	integer	Edge ID
`source_id`	integer	Source vertex ID
`target_id`	integer	Target vertex ID
`label`	text	Edge label
`properties`	text	JSON string of edge properties

`graph_delete_node(graph, id)` / `graph_delete_edge(graph, id)`

Delete a vertex or edge from a named graph. Deleting a vertex also removes all its incident edges. Persistence is handled automatically by the SQLite-backed graph store.

```sql
-- Delete a node (cascades to incident edges)
SELECT * FROM graph_delete_node('social', 2);

-- Delete a specific edge
SELECT * FROM graph_delete_edge('social', 1);
```

Parameter	Type	Description
`graph`	string	Named graph name
`id`	integer	Vertex ID or edge ID to delete

`cypher(graph_name, query) AS (columns)`

Apache AGE compatible openCypher query execution. Embeds a Cypher query in the SQL FROM clause, executing it against a named graph and returning results as a virtual table. Column names and types are specified in the `AS` clause.

The Cypher compiler uses `GraphPostingList` as the core abstraction — every clause transforms a posting list where each entry represents a binding row, consistent with UQA's posting-list-based architecture.

```
-- Create vertices and relationships
SELECT * FROM cypher('social', $$
    CREATE (a:Person {name: 'Alice', age: 30})-[:KNOWS]->(b:Person {name: 'Bob', age: 25})
    RETURN a.name, b.name
$$) AS (a_name agtype, b_name agtype);

-- Pattern matching with filtering
SELECT * FROM cypher('social', $$
    MATCH (p:Person)-[:KNOWS]->(friend:Person)
    WHERE p.age > 25
    RETURN p.name AS person, friend.name AS friend, p.age AS age
    ORDER BY p.name
$$) AS (person agtype, friend agtype, age agtype);

-- MERGE with ON CREATE SET
SELECT * FROM cypher('social', $$
    MERGE (p:Person {name: 'Carol'})
    ON CREATE SET p.age = 28
    RETURN p.name, p.age
$$) AS (name agtype, age agtype);

-- Variable-length paths
SELECT * FROM cypher('social', $$
    MATCH (a:Person)-[:KNOWS*1..3]->(b:Person)
    RETURN DISTINCT a.name, b.name
$$) AS (from_name agtype, to_name agtype);

-- SQL WHERE on Cypher results
SELECT person, age FROM cypher('social', $$
    MATCH (p:Person) RETURN p.name AS person, p.age AS age
$$) AS (person agtype, age agtype)
WHERE age > 25;
```

Parameter	Type	Description
`graph_name`	string	Named graph to query (must be created with `create_graph`)
`query`	string	openCypher query (typically wrapped in `$$...$$` dollar-quoting)

Supported Cypher clauses: `MATCH`, `OPTIONAL MATCH`, `CREATE`, `MERGE` (with `ON CREATE SET` / `ON MATCH SET`), `SET`, `DELETE` / `DETACH DELETE`, `RETURN` (with `ORDER BY`, `LIMIT`, `SKIP`, `DISTINCT`, `AS`), `WITH`, `UNWIND`

Supported expressions: property access (`n.prop`), function calls (`id(n)`, `labels(n)`, `type(r)`, etc.), arithmetic, comparison, `AND` / `OR` / `NOT` / `XOR`, `IN`, `IS NULL` / `IS NOT NULL`, `CASE` / `WHEN`, list/map literals, list indexing, parameters (`$param`)

Built-in Cypher functions: `id`, `labels`, `type`, `properties`, `keys`, `size`, `length`, `coalesce`, `toInteger`, `toFloat`, `toString`, `toBoolean`, `toLower`, `toUpper`, `trim`, `left`, `right`, `substring`, `replace`, `split`, `reverse`, `startsWith`, `endsWith`, `contains`, `head`, `tail`, `last`, `range`, `abs`, `ceil`, `floor`, `round`, `sign`, `rand`

`create_analyzer(name, config)` / `drop_analyzer(name)` / `list_analyzers()`

Text analysis management functions. `create_analyzer` registers a custom analyzer from a JSON configuration specifying tokenizer, token filters, and character filters. `drop_analyzer` removes a custom analyzer. `list_analyzers` returns all registered analyzers (both built-in and custom).

```sql
-- Create a custom analyzer with stemming
SELECT * FROM create_analyzer('english_stem', '{
    "tokenizer": {"type": "standard"},
    "token_filters": [
        {"type": "lowercase"},
        {"type": "stop", "language": "english"},
        {"type": "porter_stem"}
    ],
    "char_filters": []
}');

-- List all registered analyzers
SELECT * FROM list_analyzers();

-- Drop a custom analyzer
SELECT * FROM drop_analyzer('english_stem');
```

Parameter	Type	Description
`name`	string	Analyzer name
`config`	string	JSON configuration with `tokenizer`, `token_filters`, and `char_filters` arrays

Tokenizers

A tokenizer splits raw text into a sequence of tokens. Each analyzer has exactly one tokenizer.

Type	Description	Parameters
`whitespace`	Split on whitespace (`str.split()`)	(none)
`standard`	Unicode word-boundary tokenizer (`\w+` regex); keeps alphanumeric and underscore sequences	(none)
`keyword`	Emit the entire input as a single token, unmodified	(none)
`letter`	Extract runs of ASCII letters only (`[a-zA-Z]+`)	(none)
`pattern`	Split text using a regex delimiter	`pattern` (default `\W+`)
`ngram`	Split on whitespace, then generate all character n-grams per word	`min_gram` (default 1), `max_gram` (default 2)

Token Filters

Token filters transform the token stream produced by the tokenizer. Multiple filters are chained in order.

Type	Description	Parameters
`lowercase`	Convert all tokens to lowercase	(none)
`stop`	Remove stop words	`language` (default `"english"`), `custom_words` (optional list)
`porter_stem`	Apply the Porter stemming algorithm (suffix stripping)	(none)
`ascii_folding`	Fold Unicode characters to ASCII equivalents (NFKD normalization)	(none)
`synonym`	Expand tokens with synonyms; original token is kept, alternatives appended	`synonyms` (dict) or `synonyms_path` (file path); mutually exclusive
`ngram`	Generate character n-grams from each token	`min_gram` (default 2), `max_gram` (default 3), `keep_short` (default false)
`edge_ngram`	Generate prefix n-grams from each token (autocomplete/typeahead)	`min_gram` (default 1), `max_gram` (default 20)
`length`	Keep tokens within a length range	`min_length` (default 0), `max_length` (default 0 = no limit)

Character Filters

Character filters transform raw text before tokenization. Multiple character filters are chained in order.

Type	Description	Parameters
`html_strip`	Strip HTML tags and convert common HTML entities (`&`, `<`, etc.)	(none)
`mapping`	Apply string-level character replacements (longest-match-first)	`mapping` (dict of old -> new)
`pattern_replace`	Replace text matching a regular expression	`pattern`, `replacement` (default `""`)

Built-in Analyzer Presets

Four built-in analyzers are registered by default and cannot be overwritten or dropped.

Name	Pipeline	Description
`whitespace`	WhitespaceTokenizer -> LowerCaseFilter	Simple lowercase split; equivalent to `text.lower().split()`
`standard`	StandardTokenizer -> LowerCaseFilter -> ASCIIFoldingFilter -> StopWordFilter("english") -> PorterStemFilter	Full-featured analyzer for Latin-script languages
`standard_cjk`	StandardTokenizer -> LowerCaseFilter -> ASCIIFoldingFilter -> StopWordFilter("english") -> PorterStemFilter -> NGramFilter(2, 3, keep_short=True)	Standard pipeline extended with bigram/trigram generation for CJK scripts
`keyword`	KeywordTokenizer (no filters)	Entire input becomes a single token, unmodified

`DEFAULT_ANALYZER` uses the `standard` preset. When no analyzer is explicitly assigned to a field, the default analyzer is used for both indexing and searching.

JSON Configuration Format

The `create_analyzer` function accepts a JSON string with three keys:

```
{
    "tokenizer": {"type": "<tokenizer_type>", ...params},
    "token_filters": [
        {"type": "<filter_type>", ...params},
        ...
    ],
    "char_filters": [
        {"type": "<char_filter_type>", ...params},
        ...
    ]
}
```

Example — inline synonyms:

```
{
    "tokenizer": {"type": "standard"},
    "token_filters": [
        {"type": "lowercase"},
        {"type": "synonym", "synonyms": {
            "car": ["automobile", "vehicle", "auto"],
            "fast": ["quick", "speedy"]
        }}
    ]
}
```

Example — file-based synonyms:

```
{
    "tokenizer": {"type": "standard"},
    "token_filters": [
        {"type": "lowercase"},
        {"type": "synonym", "synonyms_path": "/path/to/synonyms.txt"}
    ]
}
```

The synonym file uses Solr/Elasticsearch format (one rule per line):

```
# Explicit mapping (one-directional): only "car" expands
car => automobile, vehicle, auto

# Equivalent synonyms (bidirectional): each term expands to the others
fast, quick, speedy
```

Custom analyzers are persisted to the SQLite catalog (`_analyzers` table) and automatically restored when the engine is reopened.

`set_table_analyzer(table, field, analyzer[, phase])`

Assigns a named analyzer to a specific table field with an optional phase parameter. This enables the Elasticsearch-style dual analyzer pattern where index-time and search-time processing differ.

```
-- Assign an analyzer for both index and search (default)
SELECT * FROM set_table_analyzer('products', 'body', 'english_stem');

-- Assign separate index and search analyzers
SELECT * FROM set_table_analyzer('products', 'body', 'plain_index', 'index');
SELECT * FROM set_table_analyzer('products', 'body', 'synonym_search',
'search');
```

Parameter	Type	Description
`table`	string	Target table name
`field`	string	Target text field
`analyzer`	string	Name of a registered analyzer
`phase`	string	`'index'`, `'search'`, or `'both'` (default: `'both'`)

Dual analyzer pattern: Separate index-time and search-time analyzers per field enable two synonym strategies:

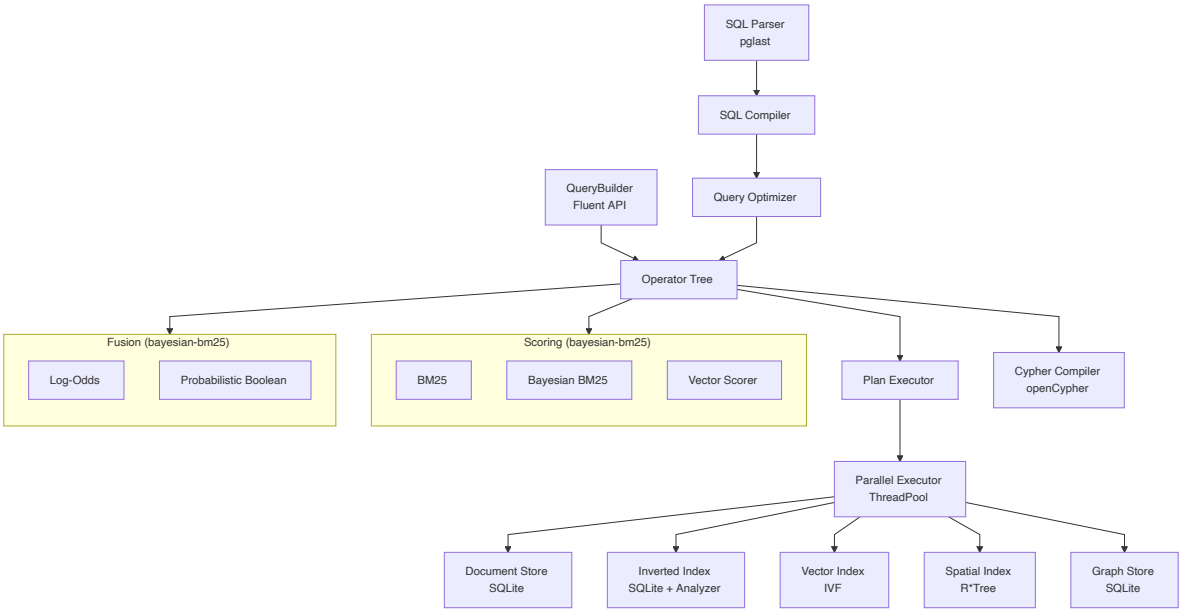
- **Search-time expansion**: Index with a plain analyzer; at query time, the search analyzer expands "car" to "car OR automobile OR vehicle". No re-indexing needed when synonyms change.
- **Index-time expansion**: Index with a synonym analyzer so "car" creates postings for "car", "automobile", and "vehicle". Queries match without expansion, but synonym changes require re-indexing.

The search-time analyzer fallback chain is: search analyzer -> index analyzer -> default analyzer.

Field-to-analyzer assignments are persisted in the `_table_field_analyzers` catalog table and automatically restored on engine restart.

5. Architecture

5.1 High-Level Architecture



5.2 Storage Layer

All storage backends have two implementations: an in-memory variant for ephemeral use and a SQLite-backed variant for persistence.

DocumentStore / SQLiteDocumentStore: Stores full documents. Per-table isolation: each SQL table gets its own `_data_{table_name}` SQLite table with typed columns.

InvertedIndex / SQLiteInvertedIndex: Maps (field, term) pairs to posting lists. Includes skip pointers for fast forward-seeking during intersection and block-max indexes for WAND pruning. Per-table, per-field SQLite tables: `_inverted_{table}_{field}`. Text analysis is handled by pluggable Analyzers (Lucene-style pipeline: CharFilter -> Tokenizer -> TokenFilter) with per-field analyzer assignment.

IVFIndex: Inverted File Index for approximate nearest neighbor search, backed by SQLite. Created explicitly via `CREATE INDEX ... USING hnsw (column)` or `USING ivf (column)`. Cosine similarity metric (vectors L2-normalized on add, similarity = dot product). Default configuration: nlist=100, nprobe=10 (customizable via `WITH (nlist, nprobe)` clause). Three-state lifecycle: UNTRAINED (brute-force scan when fewer than `max(2*nlist, 256)` vectors), TRAINED (IVF search with k-means++ centroids), STALE (automatic retrain after >20% deletes). Centroids held in memory; posting lists persisted in SQLite tables (`_ivf_centroids_{table}_{field}`, `_ivf_lists_{table}_{field}`) for instant restart without O(N log N) rebuild.

SpatialIndex: R*Tree spatial index for POINT columns. Uses SQLite's built-in R*Tree virtual table module for O(log N) bounding box queries. Each point is stored as a degenerate bounding box (min == max). Range queries use a two-pass approach: (1) coarse R*Tree bounding box filter using spherical law of cosines for accurate longitude delta, (2) fine Haversine great-circle distance verification. Created via `CREATE INDEX ... USING rtree (column)`. Per-table, per-field R*Tree tables: `_rtree_{table}_{field}`.

GraphStore / SQLiteGraphStore: Adjacency-list graph storage with indexes on (source_id, label), (target_id, label), and (label). Vertex labels stored in a dedicated indexed column for efficient label-based filtering. Vertex and edge properties stored as JSON. Named graphs use isolated `_graph_{name}_*` SQLite tables.

Catalog: SQLite-based system catalog managing metadata, table schemas, documents, graph data, vectors, postings, statistics, scoring parameters, named graphs, and custom analyzers. Write-through semantics: every mutation writes to both in-memory structures and SQLite immediately. WAL mode for concurrent reads.

5.3 SQL Compiler

The SQL compiler uses pglast (PostgreSQL parser) to parse SQL into an AST, then compiles each statement into the appropriate operation:

- **DDL**: CREATE/DROP TABLE (IF NOT EXISTS/IF EXISTS), CREATE TEMPORARY TABLE, CREATE TABLE AS SELECT, ALTER TABLE (ADD/DROP/RENAME COLUMN, SET/DROP DEFAULT, SET/DROP NOT NULL, ALTER TYPE USING), TRUNCATE TABLE, CREATE/DROP INDEX, CREATE/DROP VIEW, CREATE SEQUENCE/NEXTVAL/CURRVAL/SETVAL, ALTER SEQUENCE, TABLE name, PREPARE/EXECUTE/DEALLOCATE
- **Constraints**: PRIMARY KEY, NOT NULL, DEFAULT, UNIQUE, CHECK, FOREIGN KEY
- **DML**: INSERT (VALUES, SELECT, ON CONFLICT DO NOTHING/UPDATE, RETURNING), UPDATE (SET, FROM join, RETURNING), DELETE (WHERE, USING join, RETURNING)
- **DQL**: SELECT with DISTINCT, WHERE, GROUP BY, HAVING, ORDER BY (NULLS FIRST/LAST), LIMIT, OFFSET, FETCH FIRST n ROWS ONLY, standalone VALUES
- **Joins**: INNER, LEFT, RIGHT, FULL OUTER, CROSS JOIN with equality and non-equality ON conditions, LATERAL subquery (including correlated graph table functions)
- **Subqueries**: IN, EXISTS, scalar, correlated, LATERAL (with correlated column references in graph functions)

- **CTEs**: WITH ... AS, WITH RECURSIVE
- **Window functions**: ROW_NUMBER, RANK, DENSE_RANK, NTILE, LAG, LEAD, NTH_VALUE, PERCENT_RANK, CUME_DIST with ROWS/RANGE BETWEEN frames, WINDOW w AS (...), FILTER (WHERE ...) on window aggregates
- **Aggregates**: COUNT/SUM/AVG/MIN/MAX, STRING_AGG, ARRAY_AGG, BOOL_AND/EVERY, BOOL_OR, STDDEV, VARIANCE, PERCENTILE_CONT/DISC, MODE, JSON_OBJECT_AGG, CORR, COVAR_POP/SAMP, REGR_* (10 functions), FILTER (WHERE ...), ORDER BY within aggregate
- **Types**: INTEGER, BIGINT, SERIAL, TEXT, VARCHAR, REAL, FLOAT, DOUBLE PRECISION, NUMERIC(p,s), BOOLEAN, DATE, TIME, TIMESTAMP, TIMESTAMPTZ, INTERVAL, JSON/JSONB, UUID, BYTEA, INTEGER[] (arrays), VECTOR(N), POINT
- **JSON**: ->, ->>, #>, #>> operators, @>/<@ containment, ?/?|/?& key existence, JSONB_SET, JSONB_STRIP_NULLS, JSON_BUILD_OBJECT/ARRAY, JSON_OBJECT_KEYS, JSON_EXTRACT_PATH, JSON_TYPEOF, JSON_AGG, JSON_EACH, JSON_ARRAY_ELEMENTS
- **Date/Time**: EXTRACT, DATE_TRUNC, DATE_PART, NOW, CURRENT_DATE/TIME/TIMESTAMP, CLOCK_TIMESTAMP, TIMEOFDAY, AGE, TO_CHAR/TO_DATE/TO_TIMESTAMP, MAKE_DATE/MAKE_TIMESTAMP/MAKE_INTERVAL, TO_NUMBER, OVERLAPS, ISFINITE
- **Table functions**: GENERATE_SERIES, UNNEST, REGEXP_SPLIT_TO_TABLE, JSON_EACH/JSON_EACH_TEXT, JSON_ARRAY_ELEMENTS/JSON_ARRAY_ELEMENTS_TEXT
- **Graph functions**: cypher() (Apache AGE compatible openCypher), create_graph(), drop_graph()
- **FDW**: CREATE/DROP SERVER, CREATE/DROP FOREIGN TABLE, DuckDB FDW (Parquet/CSV/JSON), Arrow Flight SQL FDW, Hive partitioning (`hive_partitioning` option), predicate pushdown (=, !=, <, >, IN, LIKE, ILIKE, BETWEEN), full query pushdown -- entire queries on same-server foreign tables are delegated to the data source via AST deparsing, mixed foreign-local pushdown (local tables shipped as temp tables to DuckDB), LIMIT pushdown to `FDWHandler.scan()`
- **Analysis functions**: create_analyzer(), drop_analyzer(), list_analyzers(), set_table_analyzer()
- **System catalogs**: information_schema.columns, pg_catalog.pg_tables, pg_catalog.pg_views, pg_catalog.pg_indexes, pg_catalog.pg_type
- **Transactions**: BEGIN, COMMIT, ROLLBACK, SAVEPOINT
- **Utility**: EXPLAIN, ANALYZE

The compiler translates WHERE clauses into operator trees (union/intersect/complement), recognizing extended functions like `text_match`, `knn_match`, `traverse_match`, `spatial_within`, and fusion meta-functions like `fuse_log_odds`.

5.4 Query Optimizer

The optimizer applies equivalence-preserving rewrite rules:

1. **Algebraic simplification**: Applies Boolean algebra identities before any other rewriting: idempotent elimination ($A \cap A = A$, $A \cup A = A$), absorption ($A \cap (A \cup B) = A$, $A \cup (A \cap B) = A$), and empty elimination (removes empty operands from Union/Intersect).
2. **Filter pushdown**: Pushes FilterOperator through IntersectOperator to reduce intermediate result sizes early.

3. **Graph pattern filter pushdown**: Incorporates filter predicates into vertex pattern constraints, pruning during matching rather than post-filtering.
4. **Vector threshold merge**: Combines multiple vector similarity operators with the same query vector into a single search (IVF if indexed, brute-force otherwise).
5. **Intersect operand reordering**: Sorts operands by estimated cardinality (cheapest first) for optimal two-pointer intersection.
6. **Fusion signal reordering**: Sorts signals by cost estimate for early termination.
7. **R*Tree spatial index scan**: Uses the R*Tree index for POINT column range queries when available.
8. **Index scan substitution**: Replaces full-table FilterOperator scans with B-tree index scans when the estimated cost is lower.
9. **EXISTS subquery decorrelation**: Correlated `EXISTS` subqueries with equijoin predicates are decorrelated into a semi-join `FilterOperator(outer_col, InSet(values))`. The inner query executes once instead of once per outer row, yielding 116x speedup on the EXISTS benchmark.
10. **Foreign table full query pushdown**: When all tables in a query are foreign tables on the same server, the entire query (JOINs, aggregates, window functions, subqueries) is delegated to the data source via AST deparsing. For `duckdb_fdw`, local tables under 100K rows are shipped as temporary tables for mixed pushdown.

The optimizer uses a **CardinalityEstimator** backed by per-column statistics: equi-depth histograms and Most Common Values (MCVs), populated by the ANALYZE command.

5.5 Execution Engine

Plan Executor: Recursively walks the operator tree, executing each operator and collecting timing statistics. The EXPLAIN command produces a tree-formatted query plan.

Parallel Executor: Independent operator branches (children of Union, Intersect, and Fusion operators) execute concurrently via ThreadPoolExecutor. Configurable via `parallel_workers` (default: 4, 0 to disable).

Volcano Iterator Engine: Pull-based pipelined execution using Apache Arrow columnar batches (RecordBatch). Physical operators implement open/next/close. Selection vectors enable lazy filtering without materializing intermediate results. Vectorized fast paths use Arrow compute kernels (`pc.equal`, `pc.add`, `pc.sort_indices`, etc.) for simple predicates, projections, and sorts, falling back to per-row Python evaluation for complex expressions.

Disk Spilling: Blocking operators (sort, hash-aggregate, distinct) spill intermediate data to temporary Arrow IPC files when input exceeds `spill_threshold` rows:

Operator	Strategy
SortOp	Arrow-native `pc.sort_indices()` for in-memory sort; external merge sort with k-way min-heap merge for spilled runs
HashAggOp	Grace hash: partition into 16 on-disk files, aggregate each independently
DistinctOp	Hash partition dedup: same 16-partition strategy

6. Usage

6.1 Installation

```python
pip install uqa

# From source
pip install -e .

# With development dependencies
pip install -e ".[dev]"
```

Requirements: Python 3.12+, numpy >= 1.26, pyarrow >= 10.0, bayesian-bm25 >= 0.8.0, pglast >= 7.0.

6.2 Creating an Engine

```python
from uqa.engine import Engine

# In-memory (ephemeral)
engine = Engine()

# Persistent (SQLite-backed)
engine = Engine(db_path="research.db")

# With tuning
engine = Engine(
    db_path="research.db",
    parallel_workers=4,     # concurrent operator execution
    spill_threshold=100000  # spill to disk after 100K rows
)

# Context manager
with Engine(db_path="research.db") as engine:
    ...
```

6.3 SQL API

The SQL API is the primary interface for UQA.

Schema Definition

```sql
CREATE TABLE papers (
    id SERIAL PRIMARY KEY,
    title TEXT NOT NULL,
    year INTEGER NOT NULL,
    field TEXT,
    citations INTEGER DEFAULT 0,
    embedding VECTOR(64)
);
```

Supported column types: `INTEGER`, `BIGINT`, `SERIAL`, `BIGSERIAL`, `TEXT`, `VARCHAR`, `REAL`, `FLOAT`, `DOUBLE PRECISION`, `NUMERIC(p,s)`, `BOOLEAN`, `DATE`, `TIME`, `TIMESTAMP`, `TIMESTAMPTZ`, `INTERVAL`, `JSON`, `JSONB`, `UUID`, `BYTEA`, `INTEGER[]` (arrays), `VECTOR(N)`. The `VECTOR(N)` type defines an N-dimensional vector column. Vectors are stored in the document store. For fast approximate search, create an IVF index with `CREATE INDEX ... USING ivf (column)` (or `USING hnsw` for backward compatibility). Without an index, `knn_match()` uses brute-force exact cosine similarity.

Data Manipulation

```sql
INSERT INTO papers (title, year, field, citations) VALUES
    ('attention is all you need', 2017, 'NLP', 90000),
    ('bert pre-training', 2019, 'NLP', 75000),
    ('graph neural networks', 2018, 'graph', 45000);

-- UPSERT
INSERT INTO papers (id, title, year) VALUES (1, 'updated title', 2017)
ON CONFLICT (id) DO UPDATE SET title = EXCLUDED.title;

-- RETURNING
INSERT INTO papers (title, year) VALUES ('new paper', 2025) RETURNING *;

UPDATE papers SET citations = 95000
WHERE title = 'attention is all you need' RETURNING id, citations;

-- UPDATE with FROM join
UPDATE papers SET citations = s.total
FROM (SELECT paper_id, SUM(count) AS total FROM citations GROUP BY paper_id) s
WHERE papers.id = s.paper_id;

DELETE FROM papers WHERE citations < 1000 RETURNING id, title;

-- DELETE with USING join
DELETE FROM papers USING blacklist WHERE papers.id = blacklist.paper_id;
```

Relational Queries

```sql
-- Standard SQL
SELECT title, year FROM papers WHERE year >= 2018 ORDER BY year DESC LIMIT 10;

-- Aggregation
SELECT field, COUNT(*) AS cnt, AVG(citations) AS avg_cites
FROM papers GROUP BY field HAVING COUNT(*) > 1;

-- Joins (INNER, LEFT, RIGHT, FULL OUTER, CROSS)
SELECT p.title, a.name
FROM papers p INNER JOIN authors a ON p.id = a.paper_id;

-- LATERAL subquery
SELECT p.title, top.cnt
FROM papers p, LATERAL (
```

```sql
    SELECT COUNT(*) AS cnt FROM citations c WHERE c.paper_id = p.id
) top;

-- LATERAL with graph table functions (correlated column reference)
SELECT n.name, sub.edge_count
FROM nodes n, LATERAL (
    SELECT COUNT(*) AS edge_count
    FROM graph_edges('social', n.node_id, NULL, 'outgoing')
) sub;

-- Subqueries
SELECT title FROM papers
WHERE citations > (SELECT AVG(citations) FROM papers);

-- CTEs
WITH top_papers AS (
    SELECT * FROM papers WHERE citations > 50000
)
SELECT field, COUNT(*) FROM top_papers GROUP BY field;

-- Window functions
SELECT title, year,
    ROW_NUMBER() OVER (PARTITION BY field ORDER BY citations DESC) AS rank
FROM papers;

-- Views
CREATE VIEW recent_papers AS SELECT * FROM papers WHERE year >= 2020;
SELECT * FROM recent_papers;

-- Prepared statements
PREPARE find_by_year(int) AS SELECT title FROM papers WHERE year = $1;
EXECUTE find_by_year(2017);
DEALLOCATE find_by_year;

-- Transactions
BEGIN;
INSERT INTO papers (title, year) VALUES ('new paper', 2024);
SAVEPOINT sp1;
DELETE FROM papers WHERE year < 2010;
ROLLBACK TO SAVEPOINT sp1;
COMMIT;
```

Full-Text Search

```sql
-- BM25 scoring (raw scores)
SELECT title, _score FROM papers
WHERE text_match(title, 'attention transformer')
ORDER BY _score DESC;

-- Bayesian BM25 (calibrated probabilities)
SELECT title, _score FROM papers
WHERE bayesian_match(title, 'attention transformer')
ORDER BY _score DESC;
```

`text_match(field, query)` tokenizes the query, retrieves posting lists for each term, intersects them, and scores results using BM25. `bayesian_match` does the same but applies the Bayesian calibration transform, producing $P(\text{relevant}) \in [0, 1]$.

Vector Search

```
-- K-nearest neighbors with ARRAY literal
SELECT title, _score FROM papers
WHERE knn_match(embedding, ARRAY[0.1, 0.2, ...], 10) ORDER BY _score DESC;
```

```
-- K-nearest neighbors with parameter binding
qv = np.random.randn(64).astype(np.float32)
result = engine.sql(
    "SELECT title, _score FROM papers WHERE knn_match(embedding, $1, 10) ORDER BY _score DESC",
    params=[qv],
)
```

Graph Queries

```
-- Traverse: BFS from vertex 1 along 'cited_by' edges, up to 2 hops
SELECT _doc_id, title FROM traverse(1, 'cited_by', 2);

-- Regular path query: follow two 'cited_by' edges
SELECT _doc_id, title FROM rpq('cited_by/cited_by', 1);

-- Graph traversal as a scored signal in WHERE
SELECT title, _score FROM papers
WHERE traverse_match(1, 'cited_by', 2) ORDER BY _score DESC;
```

Graph Query Language (openCypher)

Apache AGE compatible graph query using the `cypher()` SQL function. Named graphs provide isolated graph namespaces.

```
-- Create a named graph
SELECT * FROM create_graph('social');

-- Create vertices with labels and relationships
SELECT * FROM cypher('social', $$
    CREATE (a:Person {name: 'Alice', age: 30})
    CREATE (b:Person {name: 'Bob', age: 25})
    CREATE (c:Person {name: 'Carol', age: 28})
    CREATE (a)-[:KNOWS]->(b)
    CREATE (a)-[:KNOWS]->(c)
    CREATE (b)-[:WORKS_WITH]->(c)
    RETURN a.name
$$) AS (name agtype);

-- Pattern matching with WHERE filtering
SELECT * FROM cypher('social', $$
    MATCH (p:Person)-[:KNOWS]->(friend:Person)
```

```
    WHERE p.age >= 28
    RETURN p.name AS person, friend.name AS friend
    ORDER BY person
$$) AS (person agtype, friend agtype);

-- MERGE: create if not exists, update if exists
SELECT * FROM cypher('social', $$
    MERGE (p:Person {name: 'Dave'})
    ON CREATE SET p.age = 35
    ON MATCH SET p.age = p.age + 1
    RETURN p.name, p.age
$$) AS (name agtype, age agtype);

-- Aggregation with WITH pipeline
SELECT * FROM cypher('social', $$
    MATCH (p:Person)-[:KNOWS]->(friend:Person)
    WITH p, count(friend) AS num_friends
    WHERE num_friends > 1
    RETURN p.name, num_friends
$$) AS (name agtype, num_friends agtype);

-- Variable-length path traversal
SELECT * FROM cypher('social', $$
    MATCH (a:Person {name: 'Alice'})-[:KNOWS*1..2]->(b:Person)
    RETURN DISTINCT b.name
$$) AS (reachable agtype);

-- OPTIONAL MATCH (left outer join semantics)
SELECT * FROM cypher('social', $$
    MATCH (p:Person)
    OPTIONAL MATCH (p)-[:MANAGES]->(e:Person)
    RETURN p.name, e.name AS manages
$$) AS (person agtype, manages agtype);

-- UNWIND lists
SELECT * FROM cypher('social', $$
    UNWIND [1, 2, 3] AS x
    RETURN x, x * x AS squared
$$) AS (x agtype, squared agtype);

-- SQL operations on Cypher results
SELECT person, age FROM cypher('social', $$
    MATCH (p:Person) RETURN p.name AS person, p.age AS age
$$) AS (person agtype, age agtype)
WHERE age > 25 ORDER BY age DESC;

-- Clean up
SELECT * FROM drop_graph('social');
```

Multi-Signal Fusion

```
-- Log-odds fusion of text, vector, and graph signals
SELECT title, _score FROM papers
WHERE fuse_log_odds(
```

```
    text_match(title, 'attention'),
    knn_match(embedding, ARRAY[0.1, 0.2, ...], 5),
    traverse_match(1, 'cited_by', 2)
) ORDER BY _score DESC;

-- Custom confidence scaling (alpha = 0.7)
SELECT title, _score FROM papers
WHERE fuse_log_odds(
    bayesian_match(title, 'neural'),
    knn_match(embedding, ARRAY[0.1, 0.2, ...], 10),
    0.7
) ORDER BY _score DESC;

-- Probabilistic Boolean
SELECT title, _score FROM papers
WHERE fuse_prob_and(
    text_match(title, 'attention'),
    knn_match(embedding, ARRAY[0.1, 0.2, ...], 5)
) ORDER BY _score DESC;

SELECT title, _score FROM papers
WHERE fuse_prob_or(
    text_match(title, 'attention'),
    knn_match(embedding, ARRAY[0.1, 0.2, ...], 5)
) ORDER BY _score DESC;
```

Hierarchical Data

```
-- Filter on nested paths
SELECT * FROM orders WHERE path_filter('shipping.city', 'Seoul');

-- Comparison operators on nested paths
SELECT * FROM orders WHERE path_filter('items.price', '>', 100);

-- Aggregate nested arrays
SELECT path_agg('items.price', 'sum') AS total FROM orders
WHERE path_filter('shipping.city', 'Seoul');

-- Access nested field values
SELECT path_value('shipping.city') AS city FROM orders;
```

Query Introspection

```
-- Show the query execution plan
EXPLAIN SELECT title, _score FROM papers
WHERE text_match(title, 'attention') ORDER BY _score DESC;

-- Collect column statistics for the optimizer
ANALYZE papers;
```

Foreign Data Wrappers

Foreign Data Wrappers (FDW) let UQA query external data sources -- Parquet files, CSV files, JSON files, S3 objects, attached DuckDB databases, and remote Arrow Flight SQL endpoints -- as if they were regular SQL tables.

Server and Foreign Table Setup

```sql
-- DuckDB FDW: in-process access to local/remote files
CREATE SERVER warehouse FOREIGN DATA WRAPPER duckdb_fdw;

-- Foreign table backed by a Parquet file
CREATE FOREIGN TABLE orders (
    order_id INTEGER,
    product_id INTEGER,
    customer TEXT,
    quantity INTEGER,
    total REAL
) SERVER warehouse OPTIONS (source '/data/orders.parquet');

-- CSV file (reader auto-detected from file extension)
CREATE FOREIGN TABLE customers (
    customer_id INTEGER,
    name TEXT,
    region TEXT
) SERVER warehouse OPTIONS (source '/data/customers.csv');

-- Hive-partitioned directory layout
CREATE FOREIGN TABLE events (
    event_id INTEGER,
    event_type TEXT,
    year INTEGER,
    month INTEGER
) SERVER warehouse OPTIONS (
    source '/data/events/',
    hive_partitioning 'true'
);

-- Arrow Flight SQL: remote SQL endpoint
CREATE SERVER analytics FOREIGN DATA WRAPPER arrow_fdw
    OPTIONS (host 'analytics.example.com', port '8815');

CREATE FOREIGN TABLE remote_sales (
    region TEXT,
    revenue REAL,
    quarter INTEGER
) SERVER analytics OPTIONS (source 'sales_summary');
```

Predicate Pushdown

WHERE predicates on foreign table columns are automatically extracted from the AST and forwarded to the FDW handler for server-side filtering. Supported operators: `=`, `!=`, `<>`, `<`, `<=`, `>`, `>=`, `IN`, `LIKE`, `NOT LIKE`, `ILIKE`, `NOT ILIKE`, `BETWEEN`. For Hive-partitioned sources, partition columns are pruned at the file-system level before any data is read.

```sql
-- Predicates pushed to DuckDB; only matching partitions are scanned
SELECT * FROM events
WHERE year = 2025 AND month IN (1, 2, 3)
ORDER BY event_id;
```

LIMIT Pushdown

The `FDWHandler.scan()` method accepts an optional `limit` parameter. When a query has a LIMIT clause with no WHERE, ORDER BY, or GROUP BY that would require full materialization, the limit is pushed down to the data source so it can stop reading early.

```sql
-- Only 10 rows fetched from DuckDB, not the full file
SELECT * FROM orders LIMIT 10;
```

Full Query Pushdown

When every table referenced in a SELECT statement -- including tables in JOINs, subqueries, CTEs, and window functions -- is a foreign table on the same server, UQA delegates the entire query to the data source instead of pulling raw rows into the UQA operator pipeline.

The pushdown mechanism:

1. Walk the AST to collect every `RangeVar` table reference (FROM, JOINs, subqueries).
2. Verify all tables resolve to foreign tables on a single server.
3. Register each foreign table as a DuckDB view pointing to its source expression (e.g., `read_parquet(...)`) or map to its Flight SQL source.
4. Deparse the original AST back to SQL via `pglast.stream.RawStream`.
5. Execute the full SQL on DuckDB (or send to the Flight SQL endpoint) and convert the Arrow result to `SQLResult`.

This means JOINs, window functions, aggregates, subqueries, ORDER BY, and LIMIT are all executed natively by DuckDB (or the Flight SQL server) in a single pass, avoiding row-by-row materialization in Python.

```sql
-- Both tables are on the same DuckDB server: entire query pushed down
SELECT o.customer,
       COUNT(*) AS num_orders,
       SUM(p.price * o.quantity) AS total_spent
FROM orders o
INNER JOIN products p ON o.product_id = p.product_id
GROUP BY o.customer
ORDER BY total_spent DESC;

-- Window functions are also pushed down
SELECT o.customer, o.order_id, o.total,
       ROW_NUMBER() OVER (PARTITION BY o.customer ORDER BY o.total DESC) AS rank
FROM orders o;

-- Subqueries pushed down as well
SELECT * FROM orders
WHERE product_id IN (SELECT product_id FROM products WHERE price > 50);
```

Full query pushdown is transparent -- no SQL syntax changes are needed. It is skipped for EXPLAIN queries (which need the UQA plan tree) and when the query references CTEs, views, or tables that are not foreign tables.

Arrow Flight SQL Pushdown

For `arrow_fdw` servers, the same full-pushdown logic applies to pure-foreign queries. The AST is deparsed to SQL, table names are replaced with their remote source expressions, and the rewritten SQL is sent to the Flight SQL endpoint as a single request. The server (Dremio, DataFusion, DuckDB Flight SQL, etc.) executes the full query and returns Arrow record batches.

```sql
-- Both tables on the same Flight SQL server: one remote round-trip
SELECT r.region, r.revenue, q.target
FROM remote_sales r
INNER JOIN remote_quotas q ON r.region = q.region
WHERE r.quarter = 4;
```

Arrow Flight SQL pushdown requires all tables in the query to be foreign tables on the same `arrow_fdw` server. Mixed queries (foreign + local tables) are not supported for Flight SQL because there is no way to ship local data to the remote endpoint.

Mixed Foreign-Local Pushdown

For `duckdb_fdw` servers, queries that reference both foreign tables and local UQA tables can still benefit from pushdown -- as long as each local table has fewer than 100,000 rows. Local tables are materialized as DuckDB temporary tables (via Arrow) before the query executes.

The process:

1. Foreign tables are registered as DuckDB views pointing to their sources.
2. Local tables are exported as Arrow tables and loaded into DuckDB as `CREATE TEMP TABLE ... AS SELECT * FROM arrow_table`.
3. The full AST is deparsed and executed on DuckDB.
4. Temporary tables and views are cleaned up after execution.

```sql
-- 'orders' and 'products' are foreign (Parquet), 'customer_notes' is local
-- The local table is shipped to DuckDB as a temp table
SELECT o.customer, p.name AS product, o.quantity, n.note
FROM orders o
INNER JOIN products p ON o.product_id = p.product_id
INNER JOIN customer_notes n ON o.customer = n.customer
ORDER BY o.order_id
LIMIT 5;
```

If any local table exceeds 100,000 rows, mixed pushdown is skipped and UQA falls back to the standard operator pipeline (per-table scans with predicate pushdown only).

Cleanup

```sql
DROP FOREIGN TABLE orders;
DROP FOREIGN TABLE products;
DROP SERVER warehouse;
```

6.4 QueryBuilder Fluent API

The fluent API provides a programmatic alternative to SQL.

```python
from uqa.engine import Engine
from uqa.core.types import Equals, GreaterThanOrEqual
import numpy as np

engine = Engine()

# All queries require a table name
q = engine.query(table="papers")

# Term search
result = q.term("attention", field="title").execute()

# BM25 scoring
result = (
    engine.query(table="papers")
    .term("attention", field="title")
    .score_bm25("attention")
    .execute()
)

# Bayesian BM25 scoring
result = (
    engine.query(table="papers")
    .term("attention", field="title")
    .score_bayesian_bm25("attention")
    .execute()
)

# Boolean composition
relevant = engine.query(table="papers").term("attention")
not_graph = engine.query(table="papers").term("graph").not_()
result = relevant.and_(not_graph).execute()

# KNN vector search
query_vec = np.random.randn(64).astype(np.float32)
result = engine.query(table="papers").knn(query_vec, k=10).execute()

# Hybrid: text AND vector
text =
engine.query(table="papers").term("transformer").score_bm25("transformer")
vector = engine.query(table="papers").knn(query_vec, k=10)
result = text.and_(vector).execute()

# Filter
result = (
    engine.query(table="papers")
    .term("attention")
    .filter("year", GreaterThanOrEqual(2020))
    .execute()
)
```

```python

# Facets
facets = engine.query(table="papers").facet("field")  # dict[value, count]

# Aggregation
count = (
    engine.query(table="papers")
    .filter("year", GreaterThanOrEqual(2020))
    .aggregate("id", "count")
)

# Graph traversal
team = engine.query(table="employees").traverse(start_id=1, label="manages", max_hops=2)
members = team.execute()
total_salary = team.vertex_aggregate("salary", "sum")

# Join
result = (
    engine.query(table="papers").term("attention")
    .join(
        engine.query(table="papers").term("transformer"),
        left_field="id",
        right_field="paper_id"
    )
    .execute()
)

# Explain
plan = engine.query(table="papers").term("attention").explain()
print(plan)
```

```python
# Sparse thresholding (ReLU-as-MAP)
result = (
    engine.query(table="papers")
    .score_bayesian_bm25("attention", "title")
    .sparse_threshold(0.3)
    .execute()
)

# Multi-field search
result = (
    engine.query(table="papers")
    .score_multi_field_bayesian("attention", ["title", "abstract"], [2.0, 1.0])
    .execute()
)

# Attention-weighted fusion
sig1 = engine.query(table="papers").score_bayesian_bm25("attention", "title")
sig2 = engine.query(table="papers").score_bayesian_bm25("transformer", "abstract")
result = engine.query(table="papers").fuse_attention(sig1, sig2).execute()
```

```python

# Learned-weight fusion
result = engine.query(table="papers").fuse_learned(sig1, sig2).execute()

# Multi-stage pipeline
s1 = engine.query(table="papers").score_bayesian_bm25("transformer", "title")
s2 = engine.query(table="papers").score_bayesian_bm25("attention", "abstract")
result = engine.query(table="papers").multi_stage([(s1, 50), (s2, 10)]).execute()

# Temporal graph traversal
result = (
    engine.query(table="social")
    .temporal_traverse(1, "knows", max_hops=2, timestamp=1700000000.0)
    .execute()
)

# GNN message passing
result = (
    engine.query(table="social")
    .message_passing(k_layers=2, aggregation="mean", property_name="score")
    .execute()
)

# Parameter learning
labels = [1, 1, 0, 0]
learned = engine.query(table="papers").learn_params("attention", labels, field="title")
```

6.5 Arrow / Parquet Export

Query results can be exported to Apache Arrow tables and Parquet files. When the result originates from the physical execution engine, Arrow RecordBatches are preserved and `to_arrow()` performs a zero-copy conversion without intermediate dict materialization.

SQL API

```python
result = engine.sql("SELECT title, year FROM papers ORDER BY year")

# Convert to Arrow Table (zero-copy when batches are available)
arrow_table = result.to_arrow()
print(arrow_table.schema)    # int64, string, etc.
print(arrow_table.num_rows)

# Write to Parquet
result.to_parquet("papers.parquet")

# Lazy iteration (generator-based, does not materialize all rows)
for row in result:
    print(row["title"])
```

QueryBuilder API

```
# Execute and return Arrow Table directly
arrow_table = (
    engine.query(table="papers")
    .term("attention", field="title")
    .score_bm25("attention")
    .execute_arrow()
)

# Execute and write to Parquet
(
    engine.query(table="papers")
    .term("attention", field="title")
    .execute_parquet("search_results.parquet")
)
```

The Arrow table from `execute_arrow()` includes `_doc_id` (int64), `_score` (float64), and any document field columns present in the posting list payload.

6.6 Programmatic Document API

The Engine provides a direct document API for adding and removing documents, graph vertices, and edges. All operations require an explicit table name — there is no global store.

```
import numpy as np
from uqa.engine import Engine
from uqa.core.types import Vertex, Edge

engine = Engine(db_path="data.db")

# Create a table first
engine.sql("""
    CREATE TABLE papers (
        id INT PRIMARY KEY,
        title TEXT NOT NULL,
        year INTEGER,
        embedding VECTOR(64)
    )
""")

# Add documents with embeddings
engine.add_document(
    doc_id=1,
    document={"title": "attention is all you need", "year": 2017},
    table="papers",
    embedding=np.random.randn(64).astype(np.float32),
)

# Delete documents
engine.delete_document(doc_id=1, table="papers")

# Graph vertices and edges require a table
```

```
engine.sql("CREATE TABLE network (id INT PRIMARY KEY, name TEXT)")
engine.add_graph_vertex(
    Vertex(vertex_id=1, properties={"name": "Alice"}), table="network"
)
engine.add_graph_vertex(
    Vertex(vertex_id=2, properties={"name": "Bob"}), table="network"
)
engine.add_graph_edge(
    Edge(
        edge_id=1, source_id=1, target_id=2,
        label="manages", properties={"since": 2020},
    ),
    table="network",
)

# Persist scoring parameters for Bayesian BM25 calibration
engine.save_scoring_params("text_signal", {"alpha": 1.0, "beta": 3.5})
params = engine.load_scoring_params("text_signal")
```

6.7 Deep Learning Methods

UQA provides analytical deep learning — CNN classifiers trained without backpropagation, using closed-form ridge regression and Product of Experts (PoE) local learning (Paper 4). Training proceeds layer by layer: convolution weights are selected via supervised grid search, and dense layer weights are computed via ridge regression. The result is a `TrainedModel` persisted to the SQLite catalog.

Layer Spec Types

Layer specifications define the architecture passed to `deep_learn()`. All spec types are immutable frozen dataclasses imported from `uqa.operators.deep_learn`.

```
from uqa.operators.deep_learn import (
    ConvSpec, PoolSpec, FlattenSpec, GlobalPoolSpec, DenseSpec, SoftmaxSpec,
)
```

Spec	Parameters	Description
`ConvSpec(kernel_hops=1, n_channels=1, init_mode="kaiming")`	`kernel_hops`: convolution neighborhood radius; `n_channels`: number of output channels (>1 uses multi-channel kernels); `init_mode`: `"kaiming"` (random normal), `"orthogonal"` (QR decomposition), `"gabor"` (structured filter bank), or `"kmeans"` (data-dependent patch dictionary)	Convolution layer
`PoolSpec(method="max", pool_size=2)`	`method`: `"max"` or `"mean"`; `pool_size`: spatial downsampling factor	Pooling layer
`FlattenSpec()`	(none)	Flatten spatial dimensions into a single vector
`GlobalPoolSpec(method="avg")`	`method`: `"avg"` (per-channel mean), `"max"` (per-channel max), or `"avg_max"` (concatenation of both, 2x channels)	Channel-preserving spatial reduction
`DenseSpec(output_channels=10)`	`output_channels`: target dimensionality (typically equal to the number of classes)	Fully connected layer (ridge regression)
`SoftmaxSpec()`	(none)	Softmax classification head

| `AttentionSpec(n_heads=1, mode="content")` | `n_heads`: number of attention heads; `mode`: `"content"` (Q=K=V=X), `"random_qk"` (random Q,K projections, V=X), or `"learned_v"` (random Q,K, supervised V via ridge regression) | Self-attention layer (Theorem 8.3, Paper 4) |

The union type `LayerSpec = ConvSpec | PoolSpec | FlattenSpec | GlobalPoolSpec | DenseSpec | SoftmaxSpec | AttentionSpec` is accepted by `deep_learn()`.

`Engine.deep_learn()`

```
Engine.deep_learn(
    model_name: str,
    table_name: str,
    label_field: str,
    embedding_field: str,
    edge_label: str,
    layer_specs: list[ConvSpec | PoolSpec | FlattenSpec | GlobalPoolSpec | DenseSpec | SoftmaxSpec | AttentionSpec],
    gating: str = "none",
    lam: float = 1.0,
    l1_ratio: float = 0.0,
    prune_ratio: float = 0.0,
) -> dict
```

Trains a CNN classifier analytically using PoE local learning with supervised convolution weight estimation. No backpropagation is used — convolution weights are selected by grid search over candidate kernels, and dense layer weights are computed via closed-form ridge regression. Each (conv, pool) stage trains an independent expert head; inference averages expert logits with accuracy-weighted PoE combination and diversity-prior shrinkage correction ($\alpha = 1/(2\sqrt{n_{experts}})$). The trained model is automatically persisted to the catalog via `save_model()`.

Parameter	Type	Description
`model_name`	str	Name for the trained model (used for persistence and prediction)
`table_name`	str	Table containing training data
`label_field`	str	Document field containing class labels
`embedding_field`	str	Document field containing embedding vectors (must be C*H*W layout)
`edge_label`	str	Edge label for graph-based convolution neighborhoods
`layer_specs`	list[LayerSpec]	Architecture specification (sequence of layer specs)
`gating`	str	Gating mode: `"none"`, `"relu"`, or `"swish"` (default: `"none"`)
`lam`	float	Ridge regression regularization parameter (default: 1.0)
`l1_ratio`	float	Elastic net L1 ratio (default: 0.0). Values > 0 enable proximal gradient descent (ISTA), warm-started from ridge.
`prune_ratio`	float	Magnitude pruning ratio (default: 0.0). Values > 0 zero the smallest weights by absolute magnitude percentile.

Returns a dict with keys:

Key	Type	Description
`model_name`	str	Name of the trained model
`training_samples`	int	Number of training samples
`num_classes`	int	Number of distinct classes
`feature_dim`	int	Dimensionality of the final feature vector
`training_accuracy`	float	Accuracy on the training set
`class_labels`	list	Sorted list of unique class labels
`l1_ratio`	float	Elastic net L1 ratio used during training
`prune_ratio`	float	Magnitude pruning ratio used during training
`weight_sparsity`	float	Fraction of zero weights in the final model

```python
import numpy as np
from uqa.engine import Engine
from uqa.operators.deep_learn import (
    ConvSpec, PoolSpec, FlattenSpec, DenseSpec, SoftmaxSpec,
)

engine = Engine()
engine.sql("CREATE TABLE digits (label INTEGER, embedding VECTOR(784))")

# Insert training data (28x28 images flattened to 784-dim vectors)
for label, pixels in training_data:
    engine.sql(
        "INSERT INTO digits (label, embedding) VALUES ($1, $2)",
        params=[label, pixels],
    )

# Train a CNN: Conv -> Pool -> Flatten -> Dense -> Softmax
result = engine.deep_learn(
    model_name="digit_classifier",
    table_name="digits",
    label_field="label",
    embedding_field="embedding",
    edge_label="spatial",
    layer_specs=[
        ConvSpec(kernel_hops=1),
        PoolSpec(method="max", pool_size=2),
        FlattenSpec(),
        DenseSpec(output_channels=10),
        SoftmaxSpec(),
    ],
    gating="relu",
    lam=1.0,
)
print(result["training_accuracy"])
```

`Engine.deep_predict()`

```
Engine.deep_predict(
    model_name: str,
    input_embedding: list[float],
) -> list[tuple[int, float]]
```

Runs inference using a trained model. Uses the same grid-accelerated forward pass as training for consistency. Per-stage expert heads produce logits that are averaged (PoE combination) with shrinkage correction, then passed through softmax to produce class probabilities.

Parameter	Type	Description
`model_name`	str	Name of a previously trained model
`input_embedding`	list[float]	Input feature vector (same dimensionality as training embeddings)

Returns a list of `(class_index, probability)` tuples sorted by descending probability.

```
# Predict class probabilities for a new image
predictions = engine.deep_predict("digit_classifier", new_image.tolist())
top_class, top_prob = predictions[0]
print(f"Predicted class: {top_class}, confidence: {top_prob:.4f}")
```

Model Catalog Methods

```
Engine.save_model(model_name: str, config: dict) -> None
Engine.load_model(model_name: str) -> dict | None
Engine.delete_model(model_name: str) -> None
```

Model persistence methods for the SQLite-backed catalog. `save_model()` stores a model configuration dict (called automatically by `deep_learn()`). `load_model()` retrieves a model by name, returning `None` if not found. `delete_model()` removes a model from both the in-memory cache and the catalog.

```
# Manual model management
engine.save_model("my_model", {"weights": [1.0, 2.0], "bias": [0.1]})
config = engine.load_model("my_model")
engine.delete_model("my_model")
```

6.8 Interactive SQL Shell

UQA includes an interactive SQL shell with syntax highlighting and autocompletion.

```
usql                # In-memory
usql --db mydata.db  # Persistent database
```

Shell commands:

Command	Description
`\dt`	List tables (regular and foreign)
`\d <table>`	Describe table schema (regular or foreign)
`\di`	List inverted-index fields per table
`\dF`	List foreign tables (server, source, options)
`\dS`	List foreign servers (type, connection options)
`\dg`	List named graphs (vertex/edge counts)
`\ds <table>`	Show column statistics (requires ANALYZE first)
`\x`	Toggle expanded (vertical) display
`\o [file]`	Redirect output to file (no arg restores stdout)
`\timing`	Toggle query timing display
`\reset`	Reset the engine
`\?`	Show help
`\q`	Quit

6.9 Transactions

UQA supports ACID transactions with savepoints through the SQL interface and the programmatic API.

```python
# SQL transactions
engine.sql("BEGIN")
engine.sql("INSERT INTO papers (title, year) VALUES ('new paper', 2024)")
engine.sql("SAVEPOINT sp1")
engine.sql("DELETE FROM papers WHERE year < 2010")
engine.sql("ROLLBACK TO SAVEPOINT sp1")  # undo the DELETE
engine.sql("COMMIT")

# Programmatic transactions
txn = engine.begin()
try:
    engine.add_document(1, {"title": "test"})
    txn.commit()
except Exception:
    txn.rollback()
```

7. Significance

7.1 Unification Through Algebra

The central contribution of UQA is demonstrating that five historically separate data paradigms can be unified under a single algebraic structure without sacrificing the expressivity of any individual paradigm. The posting list abstraction is not a lowest-common-denominator compromise but a genuine isomorphism: every operation in the Boolean algebra of posting lists has a well-defined semantics in each paradigm, and cross-paradigm compositions inherit the algebraic properties (commutativity, associativity, distributivity) of the underlying Boolean algebra.

This means, for example, that a query combining BM25 text scoring with KNN vector search and graph traversal is not an ad-hoc pipeline but a composition of operators whose algebraic properties are formally proven. The query optimizer can apply rewriting rules (filter pushdown, operand reordering) with guaranteed semantic preservation because the rules are derived from the algebraic structure, not from heuristics.

7.2 Closing the Probabilistic Relevance Gap

For nearly fifty years, BM25 scores — despite originating from a probabilistic model — have been treated as opaque numbers rather than probabilities. Bayesian BM25 closes this gap by returning BM25 to the probability space from which it was derived. The practical consequence is that BM25 scores become directly comparable with and combinable with other probabilistic signals (vector similarity, graph reachability) without ad-hoc normalization.

The log-odds conjunction framework resolves a fundamental problem with naive probabilistic combination: the conjunction shrinkage problem, where multiplying independent probability estimates drives the combined score toward zero as more signals agree. The log-odds mean with $\sqrt{n}$ confidence scaling preserves the direction of evidence while amplifying its magnitude proportionally to the number of agreeing signals.

7.3 Neural Structure from Probability

The fourth paper reveals that the multi-signal fusion pipeline — calibrate, transform to log-odds, aggregate, apply sigmoid — has the exact structure of a feedforward neural network. This is not an analogy; the correspondence is proven algebraically. The result reverses the conventional direction of explanation in neural network theory: rather than designing a neural architecture and analyzing it probabilistically, UQA begins with probability and arrives at neural structure as a mathematical consequence.

This has implications beyond information retrieval. The derivation provides probabilistic origins for the dominant activation functions (sigmoid, ReLU, Swish, GELU), shows that attention is Logarithmic Opinion Pooling with context-dependent reliability, and establishes that depth arises from iterated marginalization over latent variables.

8. Examples

Example scripts are provided in the `examples/` directory:

Fluent API (`examples/fluent/`)

Script	Topics
`text_search.py`	BM25, Bayesian BM25, boolean composition, facets, aggregation
`vector_and_hybrid.py`	KNN, hybrid search, vector exclusion, log-odds fusion
`graph.py`	Traversal, RPQ, pattern matching, graph indexes
`hierarchical.py`	Nested data, path filters, path aggregation
`analysis.py`	Text analysis pipeline, tokenizers, filters, stemming, per-field analyzers
`export.py`	Arrow/Parquet export from fluent queries, BM25 scoring, round-trip
`scoring.py`	Bayesian BM25, sparse threshold, multi-field, external priors
`fusion_advanced.py`	Attention/learned fusion, multi-stage pipelines
`graph_advanced.py`	Temporal traversal, message passing, path index, graph delta

SQL (`examples/sql/`)

Script	Topics
`basics.py`	DDL, DML, SELECT, CTE, window functions, transactions, views
`functions.py`	text_match, knn_match, path_agg, path_value, path_filter
`graph.py`	FROM traverse/rpq, aggregates, GROUP BY, WHERE
`fusion.py`	fuse_log_odds, fuse_prob_and/or/not, EXPLAIN
`analysis.py`	Text analyzers via SQL: create, list, drop, persistence
`export.py`	Arrow/Parquet export from SQL queries, type preservation, JOIN export
`spatial.py`	Geospatial: POINT, R*Tree, spatial_within, ST_Distance, ST_DWithin, fusion
`fdw.py`	Foreign Data Wrappers, DuckDB (Parquet/CSV/JSON), JOINs, Hive partitioning, full query pushdown, mixed foreign-local pushdown
`scoring_advanced.py`	Sparse threshold, multi-field match, attention/learned fusion, staged retrieval
`calibration.py`	ECE, Brier score, reliability diagram, parameter learning
`temporal_graph.py`	Temporal traversal, message passing, graph embeddings
`graph_delta.py`	Delta operations, path index invalidation, versioned rollback

9. Testing

Tests use pytest with hypothesis for property-based testing.

```
# Run all tests
python -m pytest uqa/tests/ -v

# Run a specific test file
python -m pytest uqa/tests/test_sql.py -v
```

2597 tests across 78 test files covering core algebra, operators, scoring, fusion, SQL compilation, physical execution, joins, graph operations, openCypher graph queries, SQLite persistence, transactions, cost optimization, parallel execution, PostgreSQL 17 compatibility, text analysis pipeline, Arrow/Parquet export, Foreign Data Wrappers (Hive partitioning, predicate pushdown), interactive SQL shell, calibration metrics, parameter learning, multi-field scoring, attention/learned fusion, multi-signal WAND, multi-stage retrieval, external priors, sparse thresholding, path index, graph delta, temporal graphs, GNN message passing, and end-to-end integration.

Additionally, 269 benchmarks across 13 files (in the `benchmarks/` directory) measure performance of posting list operations, storage backends, SQL compilation, physical execution, DPccp join enumeration, BM25/vector/fusion scoring, graph traversal and pattern matching, and end-to-end SQL queries. Install with `pip install -e ".[benchmark]"` and run with `python -m pytest benchmarks/ --benchmark-sort=name`.

10. References

1. Jeong, J. (2023). A Unified Mathematical Framework for Query Algebras Across Heterogeneous Data Paradigms.
2. Jeong, J. (2024). Extending the Unified Mathematical Framework to Support Graph Data Structures.
3. Jeong, J. (2026). Bayesian BM25: A Probabilistic Framework for Hybrid Text and Vector Search.
4. Jeong, J. (2026). From Bayesian Inference to Neural Computation: The Analytical Emergence of Neural Network Structure from Probabilistic Relevance Estimation.